Predicting the Initial Conditions of a Black Hole Using Simulated Hawking Radiation Data

Tanner Houser
12th Grade
Science Fair 2026
Sargent High School

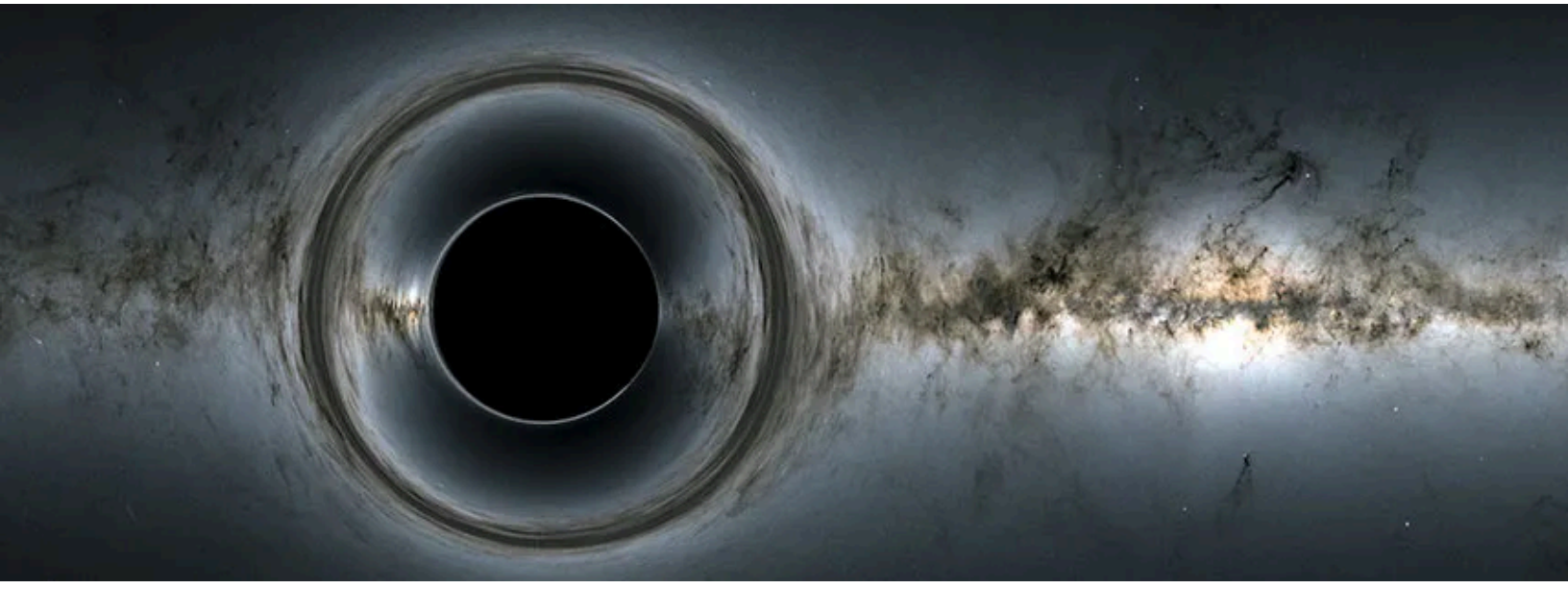


Table of Contents

Abstract.....2
Research Question.....3
Engineering Goal.....4
Rationale.....5
Materials.....7
Methodology.....8
Constraints & Limitations.....9
Program Changelog & Notes.....10
Generated Data.....81
Applications.....99
Conclusions.....100
Recommendations.....101
Resources.....102

Abstract

This project modeled the effects of Hawking radiation on black hole evolution and investigated whether black hole initial conditions can be inferred from radiation temperature data. A numerical simulation was developed in Python to reproduce black hole evaporation dynamics using recursive equations to compute mass, temperature, radius, and spin across discrete timesteps.

The model calculates decay time by integrating the evaporation process until mass approaches zero, with visualization tools implemented to analyze temporal behavior. Rotational effects were incorporated using the Kerr solutions to Einstein's field equations, and Gaussian noise was added to approximate realistic observational uncertainty.

An inverse modeling prediction algorithm was constructed to estimate the initial black hole mass by comparing simulated radiation emission curves against generated reference data across a bounded search range.

The results demonstrate that the initial mass of a non-rotating black hole can be estimated using radiation emission data within the constraints of the numerical simulation model.

Research Question

Is it possible to recover information lost in a black hole by analyzing its Hawking radiation emission spectrum?

Engineering Goal

Create a simulated model that produces realistic Hawking radiation data and demonstrate how the information lost in a black hole could be recovered using its emission spectra.

Rationale

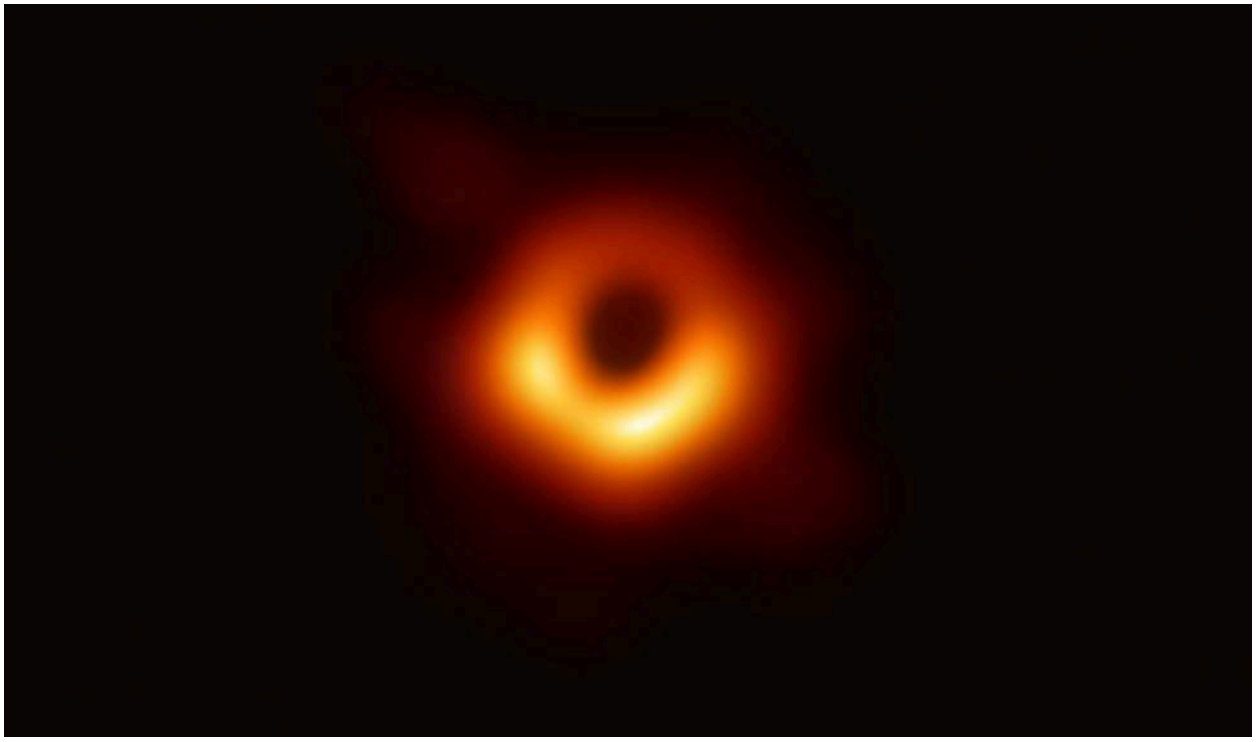
When a star collapses, its expanding energy and subsequent pressure dissipates, leading to an unimaginable amount of matter being condensed into a singular point. This is a black hole. Black holes are arguably the most mysterious and least understood objects in our universe. They contain so much gravitational attraction that not even light can escape beyond their event horizon.

Since their conception, black holes were thought to have zero entropy, meaning they contained no information or randomness. This is in direct contradiction to the law of conservation of information, however, which states that quantum information cannot be created or destroyed. This idea was so absurd that even Albert Einstein, who initially demonstrated them using his theories of relativity and gravitation in 1916, doubted their existence.

This problem would remain unsolved for nearly half a century. In 1965, Roger Penrose attempted to tackle this issue by speculating how information and energy could be extracted from a black hole using what is known as the Penrose process. However, the process known today as Hawking Radiation would not be realized until 1974, with the publication of Stephen Hawking's paper, "Black Hole Explosions?" Hawking radiation, which can be summarized with the equation $T=\hbar c3/8\pi GMk$, describes a process where quantum fluctuations in spacetime near

the event horizon of a black hole cause it to radiate energy, eventually losing its mass over an incredibly large amount of time.

While initially theoretical, black holes and Hawking radiation have both been extensively shown to be real phenomena. In 2021, researchers at MIT demonstrated the effects of Hawking radiation using sonic black holes, and in 2019, NASA released the first image of a black hole, an achievement thought impossible. My project aimed to predict the mass of black holes using simulated Hawking radiation spectral data to demonstrate the conservation of information.



A picture of the black hole at the center of the M85 galaxy, taken by the Event Horizon telescope array by NASA's Jet Propulsion Laboratory (jpl.nasa.gov) in 2019; It was the first captured image of a black hole.

Materials

1. Samsung Galaxy Book 4 Pro
2. USB-C Charger for charging laptop
3. Access to internet connection
4. VSCode (Microsoft IDE)

Methodology

- The program runs on Python scripts within a Jupyter notebook and utilizes the libraries **scipy, numpy, matplotlib, math,** and **random.**
- It can theoretically compute any data curve; however realistic simulation is limited by computational ability.
- Assume all calculations use S.I. units unless otherwise stated.

Constraints & Limitations

- Both models use standard 64-bit float variables, which can store data with limited precision.
- **Simulation Model**
    - The simulation model is primarily limited by the **precision** variable, which can be adjusted to increase/decrease data accuracy, at the cost of slower/faster compute speeds (respectively).
    - The simulation model does not have any edge case failures or scope limitations (besides at very small/large values, where overflow errors caused by Python limitations creep up, or at masses under ~1,000 kg @ low precisions, where decay time is so fast it renders as 0 seconds).
- **Prediction Model**
    - The prediction model only accounts for mass, assuming spin = 0.
    - The prediction model's precision can be adjusted using **grain,** however its accuracy tapers off and flatlines around 90%.
        - Apart from accuracy flatlining, the prediction model does not have any apparent scope limitations of edge cases (except the ones found in the simulation model due to Python limits).

Program Changelog

Version 1 Code
Jan. 26, 2026

```
# Jan. 26, 2026

import scipy.constants as cst

def temp(mass):
   T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
   return(T)

def radius(mass):
   R = (2 * cst.G * mass)/(cst.c**2)
   return(R)

def SA(r):
   SA = 4 * cst.pi * (r ** 2)
   return SA

def energyloss(temp, SA):
   P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
   return(P)

def simenergyloss(mass):
   e = energyloss(temp(mass), SA(radius(mass)))
   return(e)

def energymass(energy):
   m = energy/(cst.c ** 2)
   return(m)

# The mass of Sagittarius A*
print(str(temp(8e36)) + " kelvin")
print(str(simenergyloss(8e36)) + " watts (J/s)")

def evaptime(mass):
   newm = mass
   time = 0
   while newm > 1:
      e = simenergyloss(newm)
      newm = newm - energymass(e)
      time += 1
   return(time)
blackholemass = 1e6 # kg
et = evaptime(blackholemass)

print("Evaporated in \n" + str(et) + " seconds \n" + str(et / 60) + " minutes \n" + str(et / 3600) + " hours \n" + str(et / 86400) + "
days \n")
```

Version 1 Notes
Jan. 26, 2026

- Created a simple model that runs data through sequential equations to determine the amount of time a black hole with **x** mass takes to decay due to Hawking radiation.
- Assume all equations use S.I. units unless otherwise specified
- Equations used
    - A. Hawking temperature of a Schwarzschild black hole **(temp)**
        - T(H)=(ħc^3)/(8πGMkB)
    - B. Schwarzschild radius equation **(radius)**
        - r(s)=(2GM)/(c^2)
    - C. Surface area of a sphere **(SA)**
        - SA = 4πr^2
    - D. Stefan–Boltzmann law for blackbody radiation **(energyloss)**
        - P=σAT^4
    - E. Simplified composition of Stefan–Boltzmann law for blackbody radiation combined with surface area of a sphere **(simenergyloss)**
        - P(M)=σ4πr(s)^2T(H)^4
    - F. Mass–energy equivalence from Special Relativity **(energymass)**
        - E = mc^2
- Function **(evaptime)** iterates over the equations in this order until mass = 0
    - 1. Set initial mass (ex. **blackholemass = 1e6**)
    - 2. Plug mass into equation B to find the black hole's radius
    - 3. Convert the radius of a sphere to surface area using equation C
    - 4. Plug mass into equation A to find the Hawking temperature
    - 5. Plug Hawking temperature & surface area into equation D to find energy lost per second

    - 6. Convert energy into mass with equation F
    - 7. Subtract mass from initial mass to find the new mass
    - 8. Record the total # of iterations using variable (**time)**
    - 9. Start over with the new mass
- The program outputs the amount of time passed (# of total iterations the simulation ran) in seconds, minutes, hours & days.
- The program also outputs an example of the Hawking temperature and power of Sagittarius A*, the supermassive black hole at the center of the Milky Way galaxy, solved using the program.

Bugs/Issues

- The program doesn't graph any of data, only outputting the time elapsed
- The program uses timesteps of 1 second, which can be inaccurate at precise scales (timestep approaching 0 is ideal)
- The program doesn't record any of the variables over time
- The program is incredibly inefficient, cannot compute for masses greater than ~10^7 kg (the smallest black hole we know exists is approximately 6.6 * 10^30 kg)
- The program also uses standard 64-bit float variables which can store data with limited precision

Version 2 Code
Feb. 5, 2026

```
# Feb. 4, 2026 - Feb. 5, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
import mpmath

massdata = []
tempdata = []
raddata = []
energydata = []

# Black hole initial mass
blackholemass = 1e9 #kg

# Time step (less = higher accuracy)
timestep = float(blackholemass/100) #seconds

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

def temp(mass):
    T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
    tempdata.append(T)
    return(T)

def radius(mass):
    R = (2 * cst.G * mass)/(cst.c**2)
    raddata.append(R)
    return(R)

def SA(r):
    SA = 4 * cst.pi * (r ** 2)
    return SA

def energyloss(temp, SA):
    P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
    return(P)

def simenergyloss(mass):
    e = energyloss(temp(mass), SA(radius(mass)))
    energydata.append(e)
    return(e)

def energymass(energy):
    m = energy/(cst.c ** 2)
    return(m)

# The mass of Sagittarius A*
```

```
print(str(temp(8e36)) + " kelvin")
print(str(simenergyloss(8e36)) + " watts (J/s)")

def evaptime(mass):
    newm = mass
    time = 0
    while newm > 1:
        e = simenergyloss(newm)
        newm = newm - (energymass(e) * timestep)
        if time == addmasstime: newm += addmass
        time += timestep
        massdata.append(newm)
    return(time)

et = evaptime(blackholemass)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et / 86400) + " days \n ~" + str(et / 31557945.6) + " years")

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass Over Time')

ax1.plot(massdata, color='black')
ax1.set_xlabel('Time (s)')
ax1.set_ylabel('Mass (kg)')

tempdata = tempdata[2:]
ax2 = ax1.twinx()
ax2.plot(tempdata, color='red')
ax2.set_yscale('log')
ax2.set_ylim(tempdata[0], tempdata[-1])
ax2.set_xlabel('Time (s)')
ax2.set_ylabel('Temp (K)', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Mass Over Time')

raddata = raddata[1:]
ax3.plot(raddata, color='black')
ax3.set_yscale('log')
ax3.set_ylim(raddata[0], raddata[-1])
plt.gca().invert_yaxis()
ax3.set_xlabel('Time (s)')
ax3.set_ylabel('Radius (m)')

energydata = energydata[1:]
ax4 = ax3.twinx()
ax4.plot(energydata, color='red')
ax4.set_yscale('log')
ax4.set_ylim(energydata[2], energydata[-1])
ax4.set_xlabel('Time (s)')
ax4.set_ylabel('Power (w)', color="red")
```

Version 2 Notes
Feb. 5, 2026

- Added a function to automatically change the timestep with a changing initial mass (new maximum able to compute is ~10^10 kg)
- Added the ability to add **x** mass @ **x** timestep (ex. If you wanted to throw a human sized chunk of mass into the black hole @ time = 1,000 seconds you would input **addmass = 100, addmasstime = 1000**
- Recorded mass, temp, radius, power per iteration in a data array for graphing via **massdata, tempdata, raddata,** and **energydata**
- Added a graphing function to display **(mass, temp)** over time & **(radius, power)** over time
- Added years to the output for **(evaptime)**

Bugs/Issues

- The program uses timesteps of **initial mass/100**, which helps the program run with reasonable precision up to a point; but as Hawking temperature is not a linear function, dividing the timestep by 100 still limits the program up to ~10^10 kg (large black holes have very very small Hawking temperatures so the amount of mass subtracted each timestep is very tiny leading to insane amounts of computation)
- Ideal timestep would equal 0, impossible with standard computation however I am looking into restructuring the program to use calculus instead of iterative algebra
- The program is still very inefficient, as aforementioned it cannot compute for masses greater than ~10^10 kg (the smallest black hole we know exists is approximately 6.6 * 10^30 kg)
- The program also uses standard 64-bit float variables which can store data with limited precision
- The graphing software runs into overflow errors at masses below ~10^7 kg/above ~10^10 kg and refuses to graph, I considered using more precise variables via the mpmath library however matplotlib is not compatible with mpmath variables as they are not part of standard python

Version 3 Code
Feb. 16, 2026

```
# Feb. 16, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
from mpmath import mp

mp.dps = 100

cG = mp.mpf(str(cst.G))
cc = mp.mpf(str(cst.c))
chbar = mp.mpf(str(cst.hbar))
ck = mp.mpf(str(cst.k))
cSF = mp.mpf(str(cst.Stefan_Boltzmann))

massdata = []
tempdata = []
raddata = []
energydata = []

# Black hole initial mass
blackholemass = mp.mpf('1e9') #kg

# Time step (less = higher accuracy)
timestep = mp.mpf(blackholemass/100) #seconds

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

def temp(mass):
    mass = mp.mpf(mass)
    T = (chbar * (cc ** 3))/(8 * mp.pi * cG * mass * ck)
    tempdata.append(T)
    return(T)

def radius(mass):
    mass = mp.mpf(mass)
    R = (2 * cG * mass)/(cc**2)
    raddata.append(R)
    return(R)

def SA(r):
    r = mp.mpf(r)
    SA = 4 * mp.pi * (r ** 2)
    return SA
```

```
def energyloss(temp, SA):
    temp = mp.mpf(temp)
    SA = mp.mpf(SA)
    P = (SA * cSF * (temp ** 4))
    return(P)

def simenergyloss(mass):
    mass = mp.mpf(mass)
    e = energyloss(temp(mass), SA(radius(mass)))
    energydata.append(e)
    return(e)

def energymass(energy):
    energy = mp.mpf(energy)
    m = energy/(cc ** 2)
    return(m)

# The mass of Sagittarius A*
print(str(temp(8e36)) + " kelvin")
print(str(simenergyloss(8e36)) + " watts (J/s)")

def evaptime(mass):
    newm = mass
    time = 0
    while newm > 1:
        e = simenergyloss(newm)
        newm = newm - (energymass(e) * timestep)
        if time == addmasstime: newm += addmass
        time += timestep
        massdata.append(newm)
    return(time)

et = evaptime(blackholemass)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et /
86400) + " days \n ~" + str(et / 31557945.6) + " years")

# Converting mpmath variables to regular floats so matplotlib can graph them

def normalize(array):
    normalizedarray = []
    exponent = float(mp.re(mp.floor(mp.log10(array[0]))))
    for i in array:
        normal = float(i / 10**exponent)
        normalizedarray.append(normal)
    return(normalizedarray)

def normalizedexponent(array):
    return float(mp.re(mp.floor(mp.log10(array[0]))))

tempE = normalizedexponent(tempdata)
massE = normalizedexponent(massdata)
radE = normalizedexponent(raddata)
energyE = normalizedexponent(energydata)
```

```
tempdata = normalize(tempdata)
massdata = normalize(massdata)
raddata = normalize(raddata)
energydata = normalize(energydata)

print(tempE)
print(massE)
print(radE)
print(energyE)

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass Over Time')

ax1.plot(massdata, color='black')
ax1.set_xlabel('Time (s)')
ax1.set_ylabel('Mass (kg) (x10^' + str(massE) + ')')

tempdata = tempdata[2:]
ax2 = ax1.twinx()
ax2.plot(tempdata, color='red')
ax2.set_yscale('log')
ax2.set_ylim(tempdata[0], tempdata[-1])
ax2.set_xlabel('Time (s)')
ax2.set_ylabel('Temp (K) (x10^' + str(tempE) + ')', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Mass Over Time')

raddata = raddata[1:]
ax3.plot(raddata, color='black')
ax3.set_yscale('log')
ax3.set_ylim(raddata[0], raddata[-1])
plt.gca().invert_yaxis()
ax3.set_xlabel('Time (s)')
ax3.set_ylabel('Radius (m) (x10^' + str(radE) + ')')

energydata = energydata[1:]
ax4 = ax3.twinx()
ax4.plot(energydata, color='red')
ax4.set_yscale('log')
ax4.set_ylim(energydata[2], energydata[-1])
ax4.set_xlabel('Time (s)')
ax4.set_ylabel('Power (w) (x10^' + str(energyE) + ')', color="red")
```

Version 3 Notes
Feb. 16, 2026

- I attempted to fix the graphing issue from version 2; however, it only made the code and graphing more bloated and complicated, without solving the core issue
    - I first converted every variable into the **mpmath.dps** format, which uses an external library to allow for more precise variables than standard Python floats
    - Every variable used must be in the **mpmath** format, or the program will default to float precision
    - After performing all the calculations using **mpmath** variables and storing the results in arrays, I iterated through each index using a for loop to convert each item into standard float variables
        - My method for doing this was complicated, but in essence, I used **mp.floor** and **mp.log10** to determine the exponent for scientific notation, stored it in a separate variable, and multiplied each value by 10^that exponent. I then was able to convert each "normalized" **mpmath** variable to a standard float variable and store it in a new array. Matplotlib allowed me to graph that new normalized array, without running into errors or crashing due to [tiny numbers being represented as 0 due to precision limits] or [large numbers running into overflow errors]
    - After a lot of error correction and bug fixing, the final code was less precise, more janky, and more redundant than version 2, while still running into the same issues and crashing when attempting to graph masses not in the range of ~10e7 kg > x > 10e10 kg
    - Eventually, I abandoned this idea and returned to version 2, accepting that I won't be able to graph certain ranges

Version 4 Code
Feb. 17, 2026

```
# Feb. 17, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
import math
import random

massdata = []
tempdata = []
raddata = []
energydata = []

# Black hole initial conditions
blackholemass = 8.4e36 #kg
blackholespin = 0.9 # Value must be between 0 > x > 1, 0 = no rotation

precision = 1e21 # Higher number = more accurate, longer compute time

# Time step (less = higher accuracy)
timestep = (blackholemass**3)/precision #seconds

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

def temp(mass):
    T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
    kerrT = T * ((math.sqrt(1 - blackholespin ** 2)) / (1 + math.sqrt(1 - blackholespin ** 2)))
    tempdata.append(kerrT)
    return(kerrT)

def radius(mass):
    R = (2 * cst.G * mass)/(cst.c**2)
    kerrR = R * (1 + math.sqrt(1 - blackholespin ** 2))
    raddata.append(kerrR)
    return(kerrR)

def SA(r):
    SA = 4 * cst.pi * (r ** 2)
    return SA

def energyloss(temp, SA):
    P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
    return(P)

def simenergyloss(mass):
    e = energyloss(temp(mass), SA(radius(mass)))
    energydata.append(e)
    return(e)
```

```
def energymass(energy):
    m = energy/(cst.c ** 2)
    return(m)

# The mass of Sagittarius A*
print(str(temp(8e36)) + " kelvin")
print(str(simenergyloss(8e36)) + " watts (J/s)")

def evaptime(mass):
    newm = mass
    time = 0
    while newm > 1:
        e = simenergyloss(newm)
        newm = newm - (energymass(e) * timestep)
        if time == addmasstime: newm += addmass
        time += timestep
        massdata.append(newm)
    return(time)

et = evaptime(blackholemass)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et /
86400) + " days \n ~" + str(et / 31557945.6) + " years")

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass Over Time')

ax1.plot(massdata, color='black')
ax1.invert_yaxis
ax1.set_ylim(massdata[-2], blackholemass)
ax1.set_xlabel('Time (s)')
ax1.set_ylabel('Mass (kg)')

tempdata = tempdata[2:]
ax2 = ax1.twinx()
ax2.plot(tempdata, color='red')
ax2.set_yscale('log', base=10)
ax2.set_ylim(tempdata[0], tempdata[-1])
ax2.set_xlabel('Time (s)')
ax2.set_ylabel('Temp (K)', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Mass Over Time')

raddata = raddata[1:]
ax3.plot(raddata, color='black')
ax3.set_yscale('log')
ax3.set_ylim(raddata[0], raddata[-1])
ax3.invert_yaxis()
ax3.set_xlabel('Time (s)')
ax3.set_ylabel('Radius (m)')
```

```
energydata = energydata[1:]
ax4 = ax3.twinx()
ax4.plot(energydata, color='red')
ax4.set_yscale('log')
ax4.set_ylim(energydata[2], energydata[-1])
ax4.set_xlabel('Time (s)')
ax4.set_ylabel('Power (w)', color="red")
```

Version 4 Notes
Feb. 17, 2026

- I reverted to my version 2 code due to the errors and redundancy in version 3
- I changed the timestep formulation to be exponential instead of linear, which fixed many of the issues my code has been experiencing
    - Originally, my timestep was defined as **blackholemass / 100** seconds
    - While this formula is adequate for lower masses, a black hole's decay time due to Hawking radiation grows with the cube of its mass
    - For example, a black hole with the mass of 1e10 kg takes approximately k * 1,000 times longer than one with 1e9 kg (10^3 = 1,000)
    - My new equation sets the timestep to equal **blackholemass ** 3 / precision**, with the precision variable allowing for varying computational power (I found a value of ~1e23 to be sufficient for my laptop; however, a value closer to 1e25 or greater would be ideal)
    - The new timestep formula allows for any mass to be calculated to specified precision, allowing me to model realistic black holes like that of Sagittarius A*
    - The new timestep formula also fixed the issue I had with graphing; I suspect the real issue was not the magnitude of the numbers, but the sheer quantity, as at higher values, the array index system might've reached an overflow error
- I also included a spin parameter for the black holes, as most real black holes are spinning rapidly (higher spin = greater surface area, which causes Hawking radiation to be of a lower energy and the black hole to subsequently evaporate more slowly)
- New equations (Kerr solution, a = spin parameter)
    - Kerr temperature **(kerrT)**
        - T(H) * [(1 + sqrt(1 - a^2)] / [1 + (1 + sqrt(1 - a^2)]
    - Kerr radius **(kerrR)**
        - R * (1 + sqrt(1 - a^2)

Bugs/Issues

- Precision has been greatly reduced in favor of computability; however, infinite precision is still theoretically possible using my model
- The program still uses standard 64-bit float variables, which can store data with limited precision
- The program does not calculate or save angular momentum data over time; the spin equations and implementation could be refined a little at this point
- I have yet to implement a regression algorithm to predict initial conditions

Version 5 Code
Feb. 17, 2026

```
# Feb. 17, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
import math
import random

massdata = []
tempdata = []
raddata = []
energydata = []
spindata = []

# Black hole initial conditions
blackholemass = 8.4e36 #kg
blackholespin = 0.9 # Value must be between 0 > x > 1, 0 = no rotation

precision = 1e21 # Higher number = more accurate, longer compute time

# Time step (less = higher accuracy)
timestep = (blackholemass**3)/precision #seconds

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

def temp(mass):
    T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
    kerrT = T * ((math.sqrt(1 - blackholespin ** 2)) / (1 + math.sqrt(1 - blackholespin ** 2)))
    tempdata.append(kerrT)
    return(kerrT)

def radius(mass):
    R = (2 * cst.G * mass)/(cst.c**2)
    kerrR = R * (1 + math.sqrt(1 - blackholespin ** 2))
    raddata.append(kerrR)
    return(kerrR)

def SA(r):
    SA = 4 * cst.pi * (r ** 2)
    return SA

def energyloss(temp, SA):
    P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
    return(P)

def simenergyloss(mass):
    e = energyloss(temp(mass), SA(radius(mass)))
    energydata.append(e)
```

```
    return(e)

def energymass(energy):
    m = energy/(cst.c ** 2)
    return(m)

def angmomspin(J, mass):
    s = (cst.c * J) / (cst.G * mass ** 2)
    return(s)

def spinloss(mass, spin):
    J = energymass(simenergyloss(mass)) * ((spin * cst.G * mass) / cst.c)
    s = angmomspin(J, mass)
    return(s)

def evaptime(mass, spin):

    newm = mass
    news = spin
    time = 0

    while newm > 1:
        e = simenergyloss(newm)
        s = spinloss(newm, news)

        newm = newm - (energymass(e) * timestep)
        news = news - (s * timestep)

        if time == addmasstime: newm += addmass
        time += timestep

        massdata.append(newm)
        spindata.append(news)
    return(time)

et = evaptime(blackholemass, blackholespin)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et /
86400) + " days \n ~" + str(et / 31557945.6) + " years")

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass Over Time')

ax1.plot(massdata, color='black')
ax1.invert_yaxis
ax1.set_ylim(0, blackholemass)
ax1.set_xlabel('Time (s)')
ax1.set_ylabel('Mass (kg)')

ax2 = ax1.twinx()
ax2.plot(spindata, color='red')
ax2.set_ylim(0, 1)
ax2.set_xlabel('Time (s)')
```

```
ax2.set_ylabel('Spin (dimensionless)', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Mass Over Time')

raddata = raddata[1:]
ax3.plot(raddata, color='black')
ax3.set_ylim(raddata[0], raddata[-1])
ax3.invert_yaxis()
ax3.set_xlabel('Time (s)')
ax3.set_ylabel('Radius (m)')

tempdata = tempdata[2:]
ax4 = ax3.twinx()
ax4.plot(tempdata, color='red')
ax4.set_ylim(tempdata[0], tempdata[-1])
ax4.set_xlabel('Time (s)')
ax4.set_ylabel('Temp (K)', color="red")
```

Version 5 Notes
Feb. 17, 2026

- In version 5, I primarily focused on refining my spin functions
- In the previous version, I included spin but left it constant for simplicity; in the new model, it is iterated with every timestep and affected by the new mass and new spin each time
- I also adjusted the graphing so the first chart includes spin, and all variables are linear to show relative change with time
- The new set of equations used is as follows:
    - **Adjusted to account for the Kerr spin solutions**
        - A. Hawking temperature of a Schwarzschild black hole **(temp)**
            - T(H)=(ħc^3)/(8πGMkB) * [(1 + sqrt(1 - a^2)] / [1 + (1 + sqrt(1 - a^2)]
        - B. Schwarzschild radius equation **(radius)**
            - r(s)=(2GM)/(c^2) * (1 + sqrt(1 - a^2)
    - C. Surface area of a sphere **(SA)**
        - SA = 4πr^2
    - D. Stefan–Boltzmann law for blackbody radiation **(energyloss)**
        - P=σAT^4
    - E. Simplified composition of Stefan–Boltzmann law for blackbody radiation combined with surface area of a sphere **(simenergyloss)**
        - P(M)=σ4πr(s)^2T(H)^4
    - F. Mass–energy equivalence from Special Relativity **(energymass)**
        - E = mc^2
    - **New equations used**
        - G. Conversion between angular momentum and dimensionless spin **(angmomspin)**
            - a. = cJ/(GM^2)

        - H. Calculation of spin lost per second using change in mass and a. **(spinloss)**
            - $\Delta a. = \Delta m(Gma.)/c$

- Function **(evaptime)** now iterates over the equations in this order until mass = 0
    - 1. Set initial mass (ex. **blackholemass = 1e6**)
    - 2. Set initial spin (ex. **blackholespin = 0.9**)
    - 2. Plug mass & spin into equation B to find the black hole's radius
    - 3. Convert the radius of a sphere to surface area using equation C
    - 4. Plug mass & spin into equation A to find the Hawking temperature
    - 5. Plug Hawking temperature & surface area into equation D to find energy lost per second
    - 6. Convert energy into mass with equation F
    - 7. Subtract mass from the initial mass to find the new mass
    - 8. Using equations G & H, calculate spin lost based on mass lost
    - 8. Subtract spin lost from original spin
    - 8. Record the total # of iterations using variable (**time)**
    - 9. Start over with the new mass

Bugs/Issues

- Precision has been greatly reduced in favor of computability; however, infinite precision is still theoretically possible using my model
- The program uses standard 64-bit float variables, which can store data with limited precision
- I have yet to implement a noise model or regression algorithm to predict initial conditions

Version 6 Code
Feb. 18, 2026

```
# Feb. 18, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
import math
import random

massdata = []
tempdata = []
raddata = []
energydata = []
spindata = []

# Initial conditions
blackholemass = 8.4e36 #kg
blackholespin = 0.9 # Value must be between 0 > x > 1, 0 = no rotation
noise = 0.05 # Gaussian distribution
precision = 1e21 # Higher number = more accurate, longer compute time

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

# Time step (less = higher accuracy)
timestep = (blackholemass**3)/precision #seconds

def temp(mass):
    T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
    kerrT = T * ((math.sqrt(1 - blackholespin ** 2)) / (1 + math.sqrt(1 - blackholespin ** 2)))
    tempdata.append(kerrT)
    return(kerrT)

def radius(mass):
    R = (2 * cst.G * mass)/(cst.c**2)
    kerrR = R * (1 + math.sqrt(1 - blackholespin ** 2))
    raddata.append(kerrR)
    return(kerrR)

def SA(r):
    SA = 4 * cst.pi * (r ** 2)
    return SA

def energyloss(temp, SA):
    P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
    return(P)

def simenergyloss(mass):
    e = energyloss(temp(mass), SA(radius(mass)))
    energydata.append(e)
```

```
    return(e)

def energymass(energy):
    m = energy/(cst.c ** 2)
    return(m)

def angmomspin(J, mass):
    s = (cst.c * J) / (cst.G * mass ** 2)
    return(s)

def spinloss(mass, spin):
    J = energymass(simenergyloss(mass)) * ((spin * cst.G * mass) / cst.c)
    s = angmomspin(J, mass)
    return(s)

def evaptime(mass, spin):

    newm = mass
    news = spin
    time = 0

    while newm > 1:
        e = simenergyloss(newm)
        s = spinloss(newm, news)

        newm = newm - ((energymass(e) * timestep) * max(random.gauss(1.0, noise), 0))
        news = news - ((s * timestep) * max(random.gauss(1.0, noise), 0))

        if time == addmasstime: newm += addmass
        time += timestep

        massdata.append(newm)
        spindata.append(news)
    return(time)

et = evaptime(blackholemass, blackholespin)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et /
86400) + " days \n ~" + str(et / 31557945.6) + " years")

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass Over Time')

ax1.plot(massdata, color='black')
ax1.invert_yaxis
ax1.set_ylim(0, blackholemass)
ax1.set_xlabel('Time (s)')
ax1.set_ylabel('Mass (kg)')

ax2 = ax1.twinx()
ax2.plot(spindata, color='red')
ax2.set_ylim(0, 1)
ax2.set_xlabel('Time (s)')
```

```
ax2.set_ylabel('Spin (dimensionless)', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Mass Over Time')

raddata = raddata[1:]
ax3.plot(raddata, color='black')
ax3.set_ylim(raddata[0], raddata[-1])
ax3.invert_yaxis()
ax3.set_xlabel('Time (s)')
ax3.set_ylabel('Radius (m)')

tempdata = tempdata[2:]
ax4 = ax3.twinx()
ax4.plot(tempdata, color='red')
ax4.set_ylim(tempdata[0], tempdata[-1])
ax4.set_xlabel('Time (s)')
ax4.set_ylabel('Temp (K)', color="red")
```

Version 6 Notes
Feb. 18, 2026

- In version 6, I added a basic noise function to make the simulation more realistic, along with tightening up a few small things and adding more comments
- I first used a standard random number generator to add randomness to the simulation, but changed it to a Gaussian distribution, which is more realistic for the noise present in Hawking radiation

Bugs/Issues

- Precision has been greatly reduced in favor of computability; however, infinite precision is still theoretically possible using my model
- The program still uses standard 64-bit float variables, which can store data with limited precision

Version 7 Code
Feb. 18, 2026

```
# Feb. 18, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
import math
import random

massdata = []
tempdata = []
raddata = []
spindata = []

# Initial conditions
blackholemass = 8.6e36 #kg
blackholespin = 0.9 # Value must be between 0 > x > 1, 0 = no rotation
noise = 0.05 # Gaussian distribution
precision = 1e21 # Higher number = more accurate, longer compute time

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

# Time step (less = higher accuracy)
timestep = (blackholemass**3)/precision #seconds

def temp(mass, spin):
    T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
    kerrT = T * ((math.sqrt(1 - spin ** 2)) / (1 + math.sqrt(1 - spin ** 2)))
    return(kerrT)

def radius(mass, spin):
    R = (2 * cst.G * mass)/(cst.c**2)
    kerrR = R * (1 + math.sqrt(1 - spin ** 2))
    return(kerrR)

def SA(r):
    SA = 4 * cst.pi * (r ** 2)
    return SA

def energyloss(temp, SA):
    P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
    return(P)

def simenergyloss(mass, spin):
    e = energyloss(temp(mass, spin), SA(radius(mass, spin)))
    return(e)

def energymass(energy):
    m = energy/(cst.c ** 2)
```

```
    return(m)

def angmomspin(J, mass):
    s = (cst.c * J) / (cst.G * mass ** 2)
    return(s)

def spinloss(mass, spin):
    J = energymass(simenergyloss(mass, spin)) * ((spin * cst.G * mass) / cst.c)
    s = angmomspin(J, mass)
    return(s)

def evaptime(mass, spin):

    newm = mass
    news = spin
    time = 0

    while newm > 1:
        e = simenergyloss(newm, news)
        s = spinloss(newm, news)
        tempdata.append(temp(newm, news))

        newm = newm - ((energymass(e) * timestep) * max(random.gauss(1.0, noise), 0))
        news = news - ((s * timestep) * max(random.gauss(1.0, noise), 0))

        if time == addmasstime: newm += addmass
        time += timestep

        massdata.append(newm)
        spindata.append(news)
        raddata.append(radius(newm, news))
    return(time)

et = evaptime(blackholemass, blackholespin)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et /
86400) + " days \n ~" + str(et / 31557945.6) + " years")

def graphupperlim(x):
    return float(10 ** math.ceil(math.log10(x)))

print(type(graphupperlim(blackholemass)))

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass/Spin Over Time')

ax1.plot(massdata, color='black')
ax1.invert_yaxis
ax1.set_ylim(0, graphupperlim(blackholemass))
ax1.set_xlabel('Time (s)')
ax1.set_ylabel('Mass (kg)')

ax2 = ax1.twinx()
```

```
ax2.plot(spindata, color='red')
ax2.set_ylim(0, 1)
ax2.set_xlabel('Time (s)')
ax2.set_ylabel('Spin (dimensionless)', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Radius/Temp Over Time')

raddata = raddata[1:]
ax3.plot(raddata, color='black')
ax3.set_ylim(max(raddata), raddata[-1])
ax3.invert_yaxis()
ax3.set_xlabel('Time (s)')
ax3.set_ylabel('Radius (m)')

tempdata = tempdata[2:]
ax4 = ax3.twinx()
ax4.plot(tempdata, color='red')
ax4.set_ylim(tempdata[0], tempdata[-1])
ax4.set_xlabel('Time (s)')
ax4.set_ylabel('Temp (K)', color="red")
```

Version 7 Notes
Feb. 18, 2026

- I fixed a few bugs and glitches I found in the code
    - The arrays **tempdata[]** and **raddata[]** had 2x as many values as the other arrays in the program
        - I found that this error was being caused by the placement of the append functions; they were embedded in 2 other functions that were being called twice, one time to calculate mass and another to calculate spin
        - I fixed this by placing the functions in the main **evaptime()** function since it's only called once per timestep
        - I also removed **energydata[]** since energy is directly proportional to temperature, so storing energy data is redundant, and I no longer needed the data to graph as I replaced it with **spindata[]**
    - I also added a new function **graphupperlim(),** which calculates the log ceiling of its input, allowing me to graph more cleanly and readably (ex. 7.4e7 -> 1e8, 3.4e5 -> 1e6)
    - In addition, I also realized all my previous equations were using the initial **blackholespin** instead of an iterative variable, which was causing my data to be inaccurate (fixing this issue also made my program wildly more efficient; I suspect it's because lower spin values are much easier to compute with)
    - For around half its lifetime, black holes with a high spin were increasing in radius before dropping. I thought this was a glitch, but after doing some research, I discovered that it's actually consistent with Kerr black holes
        - This is because radius is proportional to sqrt(1 - a.), if a. starts very large (let's say a. initially equals 0.999), sqrt(1 - 0.999) = 0.032, but as a. decreases with shrinking angular momentum, sqrt(1 - 0.9) = 0.1, which is larger than 0.032. As the loss in mass overtakes the loss in spin, the change in radius switches direction

Bugs/Issues

- Precision has been greatly reduced in favor of computability; however, infinite precision is still theoretically possible using my model
- The program still uses standard 64-bit float variables, which can store data with limited precision

Version 8 Code
Feb. 18, 2026

```
# Feb. 18, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
import math
import random

massdata = []
tempdata = []
raddata = []
spindata = []

# Initial conditions
blackholemass = 8.6e36 #kg
blackholespin = 0.5 # Value must be between 0 > x > 1, 0 = no rotation
noise = 0.001 # Gaussian distribution
precision = 1e21 # Higher number = more accurate, longer compute time

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

# Time step (less = higher accuracy)
timestep = (blackholemass**3)/precision #seconds

def temp(mass, spin):
    T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
    kerrT = T * ((math.sqrt(1 - spin ** 2)) / (1 + math.sqrt(1 - spin ** 2)))
    return(kerrT)

def radius(mass, spin):
    R = (cst.G * mass)/(cst.c**2)
    kerrR = R * (1 + math.sqrt(1 - spin ** 2))
    return(kerrR)

def SA(r):
    SA = 4 * cst.pi * (r ** 2)
    return SA

def energyloss(temp, SA):
    P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
    return(P)

def simenergyloss(mass, spin):
    e = energyloss(temp(mass, spin), SA(radius(mass, spin)))
    return(e)

def energymass(energy):
    m = energy/(cst.c ** 2)
```

```
    return(m)

def angmomspin(J, mass):
    s = (cst.c * J) / (cst.G * mass ** 2)
    return(s)

def spinloss(mass, spin):
    J = energymass(simenergyloss(mass, spin)) * ((spin * cst.G * mass) / cst.c)
    s = angmomspin(J, mass)
    return(s)

def evaptime(mass, spin):

    newm = mass
    news = spin
    time = 0

    while newm > 1:
        e = simenergyloss(newm, news)
        s = spinloss(newm, news)
        tempdata.append(temp(newm, news))

        newm = newm - ((energymass(e) * timestep) * max(random.gauss(1.0, noise), 0))
        news = news - ((s * timestep) * max(random.gauss(1.0, noise), 0))

        if time == addmasstime: newm += addmass
        time += timestep

        massdata.append(newm)
        spindata.append(news)
        raddata.append(radius(newm, news))
    return(time)

et = evaptime(blackholemass, blackholespin)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et /
86400) + " days \n ~" + str(et / 31557945.6) + " years")

def timescale(time):
    if time <= 60: return(0)
    elif time <= 3600: return(1)
    elif time <= 86400: return(2)
    elif time <= 31557945.6: return(3)
    else: return(4)

def time(array):
    time = []
    for i in range(len(array)):
        if timescale(et) == 0: time.append(i * timestep)
        elif timescale(et) == 1: time.append(i * timestep/60)
        elif timescale(et) == 2: time.append(i * timestep/3600)
        elif timescale(et) == 3: time.append(i * timestep/86400)
        else: time.append(i * timestep/31557945.6)
    return(time)
```

```
def timeunits(time):
    if timescale(time) == 0: return('seconds')
    elif timescale(time) == 1: return('minutes')
    elif timescale(time) == 2: return('hours')
    elif timescale(time) == 3: return('days')
    else: return('years')

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass/Spin Over Time')

ax1.plot(time(massdata), massdata, color='black')
ax1.invert_yaxis
ax1.set_ylim(0, blackholemass*1.1)
ax1.set_xlabel('Time (' + timeunits(et) + ')')
ax1.set_ylabel('Mass (kg)')

ax2 = ax1.twinx()
ax2.plot(time(spindata), spindata, color='red')
ax2.set_ylim(0, 1)
ax2.set_ylabel('Spin (dimensionless)', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Radius/Temp Over Time')

raddata = raddata[1:]
ax3.plot(time(raddata), raddata, color='black')
ax3.set_ylim(0, max(raddata)*1.1)
ax3.set_xlabel('Time (' + timeunits(et) + ')')
ax3.set_ylabel('Radius (m)')

tempdata = tempdata[2:]
ax4 = ax3.twinx()
ax4.plot(time(tempdata), tempdata, color='red')
ax4.set_ylim(tempdata[0], tempdata[-1])
ax4.set_yscale('log')
ax4.set_ylabel('Temp (K)', color="red")
```

Version 8 Notes
Feb. 18, 2026

- I fixed an issue with my radius equation, which was multiplying my calculated radius by a factor of 2
    - The normal Schwarzschild radius equation uses **2GM / c^2,** but the Kerr solution uses **GM / c^2** multiplied by the spin factor. I forgot to change it when implementing spin
- I also removed the **graphupperlim()** function and instead multiplied the max value by 1.1, to make the graphs still readable but more consistent
- I also decided to make the temperature graph logarithmic to improve readability
- I fixed an issue with graphing where the x-axis did not scale with the timestep
    - I added a new function, **time()** which returns a new array with the same length as the inputted array, but instead stores the time at each timestep
    - This should also make prediction models easier
- I also included the **timescale()** and **timeunits()** function to adjust the scale and scale for seconds/minutes/hours/years for improved readability

Bugs/Issues

- Precision has been greatly reduced in favor of computability; however, infinite precision is still theoretically possible using my model
- The program still uses standard 64-bit float variables, which can store data with limited precision

Version 9 Code
Feb. 19, 2026

```
# Feb. 19, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
import math
import random

massdata = []
tempdata = []
raddata = []
spindata = []

def resetdata():
    massdata.clear()
    tempdata.clear()
    raddata.clear()
    spindata.clear()
resetdata()

# Initial conditions

kg = 0 # Mass in kg
solarmass = 4.2e6 # Mass in M⊙ (1 M⊙ = mass of our Sun)
blackholemass = kg + solarmass * 1.988416e30

blackholespin = 0.5 # Value must be between 0 > x > 1, 0 = no rotation
noise = 0.001 # Gaussian distribution
precision = 1e21 # Higher number = more accurate, longer compute time

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

# Time step (less = higher accuracy)
timestep = (blackholemass**3)/precision #seconds

def temp(mass, spin):
    T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
    kerrT = T * ((math.sqrt(1 - spin ** 2)) / (1 + math.sqrt(1 - spin ** 2)))
    return(kerrT)

def radius(mass, spin):
    R = (cst.G * mass)/(cst.c**2)
    kerrR = R * (1 + math.sqrt(1 - spin ** 2))
    return(kerrR)

def SA(r):
    SA = 4 * cst.pi * (r ** 2)
    return SA
```

```
def energyloss(temp, SA):
    P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
    return(P)

def simenergyloss(mass, spin):
    e = energyloss(temp(mass, spin), SA(radius(mass, spin)))
    return(e)

def energymass(energy):
    m = energy/(cst.c ** 2)
    return(m)

def angmomspin(J, mass):
    s = (cst.c * J) / (cst.G * mass ** 2)
    return(s)

def spinloss(mass, spin):
    J = energymass(simenergyloss(mass, spin)) * ((spin * cst.G * mass) / cst.c)
    s = angmomspin(J, mass)
    return(s)

def evaptime(mass, spin, timestep):

    newm = mass
    news = spin
    time = 0

    while newm > 1:
        e = simenergyloss(newm, news)
        s = spinloss(newm, news)
        tempdata.append(temp(newm, news))

        newm = newm - ((energymass(e) * timestep) * max(random.gauss(1.0, noise), 0))
        news = news - ((s * timestep) * max(random.gauss(1.0, noise), 0))

        if time == addmasstime: newm += addmass
        time += timestep

        massdata.append(newm)
        spindata.append(news)
        raddata.append(radius(newm, news))
    return(time)

resetdata()
et = evaptime(blackholemass, blackholespin, timestep)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et /
86400) + " days \n ~" + str(et / 31557945.6) + " years")

def timescale(time):
    if time <= 60: return(0)
    elif time <= 3600: return(1)
    elif time <= 86400: return(2)
```

```
    elif time <= 31557945.6: return(3)
    else: return(4)

def time(array):
    time = []
    for i in range(len(array)):
        if timescale(et) == 0: time.append(i * timestep)
        elif timescale(et) == 1: time.append(i * timestep/60)
        elif timescale(et) == 2: time.append(i * timestep/3600)
        elif timescale(et) == 3: time.append(i * timestep/86400)
        else: time.append(i * timestep/31557945.6)
    return(time)

def timeunits(time):
    if timescale(time) == 0: return('seconds')
    elif timescale(time) == 1: return('minutes')
    elif timescale(time) == 2: return('hours')
    elif timescale(time) == 3: return('days')
    else: return('years')

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass/Spin Over Time')

ax1.plot(time(massdata), massdata, color='black')
ax1.invert_yaxis
ax1.set_ylim(0, blackholemass*1.1)
ax1.set_xlabel('Time (' + timeunits(et) + ')')
ax1.set_ylabel('Mass (kg)')

ax2 = ax1.twinx()
ax2.plot(time(spindata), spindata, color='red')
ax2.set_ylim(0, 1)
ax2.set_ylabel('Spin (dimensionless)', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Radius/Temp Over Time')

raddata = raddata[1:]
ax3.plot(time(raddata), raddata, color='black')
ax3.set_ylim(0, max(raddata)*1.1)
ax3.set_xlabel('Time (' + timeunits(et) + ')')
ax3.set_ylabel('Radius (m)')

tempdata = tempdata[2:]
ax4 = ax3.twinx()
ax4.plot(time(tempdata), tempdata, color='red')
ax4.set_ylim(tempdata[0], tempdata[-1])
ax4.set_yscale('log')
ax4.set_ylabel('Temp (K)', color="red")

# Initial Conditions Predictor
```

```
precision = 1e7 # Higher number = more accurate, longer compute time
gridsize = 50 # Number of variations per run
spinlim = 1 # Highest spin tested for
grain = 10 # Number of times data refined

truemassdata = massdata
truetempdata = tempdata
trueraddata = raddata
truespindata = spindata

alldata = []
alldata.clear()

def predict(data):

    bestmass = 0
    massgrid = []

    for i in range(gridsize):
        massgrid.append(10**i)

    for i in massgrid:
        print (i)
        timestep = (i**3)/precision
        print(timestep)
        resetdata()
        alldata.append(evaptime(i, 0, timestep))
        print(evaptime(i, 0, timestep))

    return(alldata)

x = predict([])
```

Version 9 Notes
Feb. 19, 2026

- Added M⊙ (solar mass) in addition to kg in initial conditions, which is the standard unit used for measuring black hole mass
    - I initially used kg because my first version could not compute even 1 M⊙ (1 M⊙ = ~1.988416e30 kg)
- I attempted to implement a prediction algorithm, but I had a scope issue trying to complete the whole thing at once, and I'm going to start from scratch in version 10
    - My idea is to implement a coarse-to-fine grid search algorithm, where I run a low-quality simulation for every value in a set range until I arrive at one that most closely matches the original data curve. Once I find the most similar curve, I zoom in and run the test again at a smaller scale to narrow down the search
    - This is much easier and simpler/more efficient than using a regression model or a machine learning black box, and I don't have to use any external libraries, as I can code the entire thing by myself

Bugs/Issues

- Precision has been greatly reduced in favor of computability; however, infinite precision is still theoretically possible using my model
- The program still uses standard 64-bit float variables, which can store data with limited precision

Version 10 Notes
Feb. 19, 2026

```
# Feb. 19, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
import math
import random

massdata = []
tempdata = []
raddata = []
spindata = []

# Initial conditions

kg = 1e10 # Mass in kg
solarmass = 0 # Mass in M⊙ (1 M⊙ = mass of our Sun)
blackholemass = kg + solarmass * 1.988416e30

blackholespin = 0.5 # Value must be between 0 > x > 1, 0 = no rotation
noise = 1 # Gaussian distribution
precision = 1e21 # Higher number = more accurate, longer compute time

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

# Time step (less = higher accuracy)
timestep = (blackholemass**3)/precision #seconds

def temp(mass, spin):
    T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
    kerrT = T * ((math.sqrt(1 - spin ** 2)) / (1 + math.sqrt(1 - spin ** 2)))
    return(kerrT)

def radius(mass, spin):
    R = (cst.G * mass)/(cst.c**2)
    kerrR = R * (1 + math.sqrt(1 - spin ** 2))
    return(kerrR)

def SA(r):
    SA = 4 * cst.pi * (r ** 2)
    return SA

def energyloss(temp, SA):
    P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
    return(P)

def simenergyloss(mass, spin):
    e = energyloss(temp(mass, spin), SA(radius(mass, spin)))
```

```
    return(e)

def energymass(energy):
    m = energy/(cst.c ** 2)
    return(m)

def angmomspin(J, mass):
    s = (cst.c * J) / (cst.G * mass ** 2)
    return(s)

def spinloss(mass, spin):
    J = energymass(simenergyloss(mass, spin)) * ((spin * cst.G * mass) / cst.c)
    s = angmomspin(J, mass)
    return(s)

def evaptime(mass, spin, ts):

    newm = mass
    news = spin
    time = 0

    while newm > 1:
        e = simenergyloss(newm, news)
        s = spinloss(newm, news)
        tempdata.append(temp(newm, news))

        newm = newm - ((energymass(e) * ts) * max(random.gauss(1.0, noise), 0))
        news = news - ((s * ts) * max(random.gauss(1.0, noise), 0))

        if time == addmasstime: newm += addmass
        time += ts

        massdata.append(newm)
        spindata.append(news)
        raddata.append(radius(newm, news))
    return(time)

resetdata()
et = evaptime(blackholemass, blackholespin, timestep)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et /
86400) + " days \n ~" + str(et / 31557945.6) + " years")

def timescale(time):
    if time <= 60: return(0)
    elif time <= 3600: return(1)
    elif time <= 86400: return(2)
    elif time <= 31557945.6: return(3)
    else: return(4)

def time(array):
    time = []
    for i in range(len(array)):
        if timescale(et) == 0: time.append(i * timestep)
```

```
        elif timescale(et) == 1: time.append(i * timestep/60)
        elif timescale(et) == 2: time.append(i * timestep/3600)
        elif timescale(et) == 3: time.append(i * timestep/86400)
        else: time.append(i * timestep/31557945.6)
    return(time)

def timeunits(time):
    if timescale(time) == 0: return('seconds')
    elif timescale(time) == 1: return('minutes')
    elif timescale(time) == 2: return('hours')
    elif timescale(time) == 3: return('days')
    else: return('years')

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass/Spin Over Time')

ax1.plot(time(massdata), massdata, color='black')
ax1.invert_yaxis
ax1.set_ylim(0, blackholemass*1.1)
ax1.set_xlabel('Time (' + timeunits(et) + ')')
ax1.set_ylabel('Mass (kg)')

ax2 = ax1.twinx()
ax2.plot(time(spindata), spindata, color='red')
ax2.set_ylim(0, 1)
ax2.set_ylabel('Spin (dimensionless)', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Radius/Temp Over Time')

raddata = raddata[1:]
ax3.plot(time(raddata), raddata, color='black')
ax3.set_ylim(0, max(raddata)*1.1)
ax3.set_xlabel('Time (' + timeunits(et) + ')')
ax3.set_ylabel('Radius (m)')

tempdata = tempdata[2:]
ax4 = ax3.twinx()
ax4.plot(time(tempdata), tempdata, color='red')
ax4.set_ylim(tempdata[0], tempdata[-1])
ax4.set_yscale('log')
ax4.set_ylabel('Temp (K)', color="red")

# Initial Conditions Predictor

truemassdata = massdata.copy()
truetempdata = tempdata.copy()
trueraddata = raddata.copy()
truespindata = spindata.copy()

precision = 1e19 # Higher number = more accurate, longer compute time
gridsize = 50 # Number of variations per run
```

```
spinlim = 1 # Highest spin tested for
grain = 10 # Number of times data refined

alldata = []
accuracydata = []

def predict(data):
    alldata.clear()
    accuracydata.clear()
    resetdata()

    truedata = data.copy()
    tempmass = 1
    tempdata = []

    while tempmass <= 1e100:
        resetdata()
        ts = (tempmass**3)/precision
        evaptime(tempmass, 0, ts)
        alldata.append(massdata.copy())
        tempmass *= 10

    for i in alldata:
        tempdata.clear()
        loc = 0
        for j in i:
            tempdata.append((truedata[loc] - j) ** 2)
            loc += 1
        accuracydata.append(tempdata.copy()[1:])

predict(truemassdata)

def bestmatch(data):

    avgdata = []
    bestmatchloc = 0

    for i in data:
        avg = 0
        for j in i:
            avg += j
        avgdata.append(avg/len(i))

    bestmatch = avgdata[0]

    loc = 0
    for i in avgdata:
        if i < bestmatch:
            bestmatch = i
            bestmatchloc = loc
            print("New best match: " + f"{10**bestmatchloc:.2e}" + "kg")
        loc+=1
    return(bestmatch, bestmatchloc)
```

```
best = bestmatch(accuracydata.copy()[1:])
dif = math.sqrt(best[0])
print('Closest match: ~' + f"{10**best[1]:e}" + 'kg\nDifference: ' + f"{dif:e}")
```

Version 10 Notes
Feb. 19, 2026

- I implemented a very basic initial mass prediction algorithm that is accurate to ~2 orders of magnitude
    - The algorithm uses a gridsort mechanism
        - The **predict()** function first simulates the temperature data curve for every magnitude in the range of 10^0 - 10^100 kg and stores each curve as an independent array object in **alldata[]**
        - It then compares each data point in each array to the original data array, calculating its similarity using **(original data - sim data)^2**
        - It stores the similarity value for each data point as an array inside the array **accuracydata[]**
        - The **bestmatch()** function then goes through each array and takes the average similarity value, storing the average for each array as an individual value in **avgdata[]**
        - It then compares each value in **avgdata[]** and finds the value closest to zero. The value closest to zero is considered the most similar data curve, and 10^values location in **avgdata[]** is displayed as the best match
    - The prediction algorithm is in its very early stages; it does not include spin (assuming spin = 0) and cannot zoom in to fine-grained results for greater accuracy. It also does not allow for custom levels of refinement and can only calculate 100 values of magnitude from 1e0 -> 1e100 kg
    - I had a lot of issues with implementing and programming the model; the main problem I experienced was caused by the way Python manages adding objects to lists. When I tried to append a list to a list, it would append the reference to the list instead of making an individual copy.

Bugs/Issues

- Precision has been greatly reduced in favor of computability; however, infinite precision is still theoretically possible using the simulator
- The simulator still uses standard 64-bit float variables, which can store data with limited precision
- The prediction algorithm is only accurate to ~1 order of magnitude, and does not allow for specified precision or accuracy
- The prediction algorithm does not include spin
- The prediction algorithm measures similarity using (original - sim)^2, which could be refined
- There are likely still many small bugs and edge case scenarios that need to be fixed in the prediction algorithm
- I would also like to graph the predicted vs actual mass data curves
- It also uses mass data instead of temperature data for simplicity; however, temperature data is much more realistic for real-world measurements

Version 11 Notes
Feb. 23, 2026

```
# Feb. 23, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
import math
import random

massdata = []
tempdata = []
raddata = []
spindata = []

def resetdata():
    massdata.clear()
    tempdata.clear()
    raddata.clear()
    spindata.clear()
resetdata()

# Initial conditions

kg = 8.6e36 # Mass in kg
solarmass = 0 # Mass in M⊙ (1 M⊙ = mass of our Sun)
blackholemass = kg + solarmass * 1.988416e30

blackholespin = 0 # Value must be between 0 > x > 1, 0 = no rotation
noise = 0 # Gaussian distribution
precision = 1e21 # Higher number = more accurate, longer compute time

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

# Time step (less = higher accuracy)
timestep = (blackholemass**3)/precision #seconds

def temp(mass, spin):
    T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
    kerrT = T * ((math.sqrt(1 - spin ** 2)) / (1 + math.sqrt(1 - spin ** 2)))
    return(kerrT)

def radius(mass, spin):
    R = (cst.G * mass)/(cst.c**2)
    kerrR = R * (1 + math.sqrt(1 - spin ** 2))
    return(kerrR)

def SA(r):
    SA = 4 * cst.pi * (r ** 2)
    return SA
```

```
def energyloss(temp, SA):
    P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
    return(P)

def simenergyloss(mass, spin):
    e = energyloss(temp(mass, spin), SA(radius(mass, spin)))
    return(e)

def energymass(energy):
    m = energy/(cst.c ** 2)
    return(m)

def angmomspin(J, mass):
    s = (cst.c * J) / (cst.G * mass ** 2)
    return(s)

def spinloss(mass, spin):
    J = energymass(simenergyloss(mass, spin)) * ((spin * cst.G * mass) / cst.c)
    s = angmomspin(J, mass)
    return(s)

def evaptime(mass, spin, ts):

    newm = mass
    news = spin
    time = 0

    while newm > 1:
        e = simenergyloss(newm, news)
        s = spinloss(newm, news)
        tempdata.append(temp(newm, news))

        newm = newm - ((energymass(e) * ts) * max(random.gauss(1.0, noise), 0))
        news = news - ((s * ts) * max(random.gauss(1.0, noise), 0))

        if time == addmasstime: newm += addmass
        time += ts

        massdata.append(newm)
        spindata.append(news)
        raddata.append(radius(newm, news))
    return(time)

resetdata()
et = evaptime(blackholemass, blackholespin, timestep)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et /
86400) + " days \n ~" + str(et / 31557945.6) + " years")

def timescale(time):
    if time <= 60: return(0)
    elif time <= 3600: return(1)
    elif time <= 86400: return(2)
```

```
    elif time <= 31557945.6: return(3)
    else: return(4)

def time(array):
    time = []
    for i in range(len(array)):
        if timescale(et) == 0: time.append(i * timestep)
        elif timescale(et) == 1: time.append(i * timestep/60)
        elif timescale(et) == 2: time.append(i * timestep/3600)
        elif timescale(et) == 3: time.append(i * timestep/86400)
        else: time.append(i * timestep/31557945.6)
    return(time)

def timeunits(time):
    if timescale(time) == 0: return('seconds')
    elif timescale(time) == 1: return('minutes')
    elif timescale(time) == 2: return('hours')
    elif timescale(time) == 3: return('days')
    else: return('years')

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass/Spin Over Time')

ax1.plot(time(massdata), massdata, color='black')
ax1.invert_yaxis
ax1.set_ylim(0, blackholemass*1.1)
ax1.set_xlabel('Time (' + timeunits(et) + ')')
ax1.set_ylabel('Mass (kg)')

ax2 = ax1.twinx()
ax2.plot(time(spindata), spindata, color='red')
ax2.set_ylim(0, 1)
ax2.set_ylabel('Spin (dimensionless)', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Radius/Temp Over Time')

raddata = raddata[1:]
ax3.plot(time(raddata), raddata, color='black')
ax3.set_ylim(0, max(raddata)*1.1)
ax3.set_xlabel('Time (' + timeunits(et) + ')')
ax3.set_ylabel('Radius (m)')

tempdata = tempdata[2:]
ax4 = ax3.twinx()
ax4.plot(time(tempdata), tempdata, color='red')
ax4.set_ylim(tempdata[0], tempdata[-1])
ax4.set_yscale('log')
ax4.set_ylabel('Temp (K)', color="red")

# Initial Conditions Predictor
```

```
truemassdata = massdata.copy()
truetempdata = tempdata.copy()
trueraddata = raddata.copy()
truespindata = spindata.copy()

alldata = []
accuracydata = []
precision = 1e19

def predict(data):
    alldata.clear()
    accuracydata.clear()
    resetdata()

    truedata = data.copy()
    tempmass = massmin
    tempdata = []

    while tempmass <= massmax:
        resetdata()
        ts = (tempmass**3)/precision
        evaptime(tempmass, 0, ts)
        alldata.append(massdata.copy())
        tempmass *= 10/grain

    for i in alldata:
        tempdata.clear()
        loc = 0
        for j in i:
            tempdata.append(abs(truedata[loc] - j) / abs(truedata[loc]))
            loc += 1
        accuracydata.append(tempdata.copy()[1:])

def bestmatch(data):

    avgdata = []
    bestmatchloc = 0

    for i in data:
        avg = 0
        for j in i:
            avg += j
        avgdata.append(avg/len(i))

    bestmatch = avgdata[0]
    print(list(enumerate(avgdata)))

    loc = 0
    for i in avgdata:
        if i < bestmatch:
            bestmatch = i
            bestmatchloc = loc - 1
            print("New best match: " + f"{massmin*(10/grain)**bestmatchloc:.2e}" + "kg")
        loc+=1
```

```
    return(bestmatch, bestmatchloc)

grain = 9.5 # Larger = more accuracy, DNE > 10
massmin = 1e55
massmax = 1e65

predict(truemassdata)
best = bestmatch(accuracydata.copy())
closestmatch = ((10/grain)**(best[1]))*massmin
print('Closest match: ~' + f"{closestmatch:e}" + 'kg\nAccuracy: ' + str(100-(abs(100-round((closestmatch/blackholemass)*100))))
+ '%')
```

Version 11 Notes
Feb. 23, 2026

- I fixed several issues with my original prediction model and greatly increased the resolution
    - I adjusted my model to account for varying logarithms, allowing finer/coarser accuracy adjustment. I also added an adjustment for range to save on compute time
        - My original model scanned every value 10^x from range [1e0, 1e100]
        - My new prediction algorithm can scan every value y^x, with y defined by 10/**grain,** from any range [a, b]
    - The new model also measures distance from true data as **abs(true - simulated) / abs (true),** which scales more accurately with larger masses/more precision
    - The new model calculates accuracy as a percentage
        - **100 - abs(100 - (prediction/true) * 100)**
    - I had a lot of issues implementing the new model; my code needs polishing and more comments to make adjustments and updates easier

Bugs/Issues

- Precision has been greatly reduced in favor of computability; however, infinite precision is still theoretically possible using the simulator
- The simulator still uses standard 64-bit float variables, which can store data with limited precision
- The prediction algorithm does not include spin
- There are likely still many small bugs and edge case scenarios that need to be fixed in the prediction algorithm
- I would also like to graph the predicted vs actual mass data curves
- It also uses mass data instead of temperature data for simplicity; however, temperature data is much more realistic for real-world measurements

Version 12 Code
Feb. 23, 2026

```
# Feb. 23, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
import math
import random

massdata = []
tempdata = []
raddata = []
spindata = []

def resetdata():
    massdata.clear()
    tempdata.clear()
    raddata.clear()
    spindata.clear()
resetdata()

# Initial conditions

kg = 8.6e36 # Mass in kg
solarmass = 0 # Mass in M⊙ (1 M⊙ = mass of our Sun)
blackholemass = kg + solarmass * 1.988416e30
blackholespin = 0 # Value must be between 0 > x > 1, 0 = no rotation
noise = 0.5 # Gaussian distribution

# Higher number = more accurate, longer compute time
realprecision = 1e21 # ~1e21 for my pc
predprecision = 1e19 # ~1e19 for my pc

precision = realprecision
timestep = 0

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

timestep = (blackholemass**3)/precision #seconds

def temp(mass, spin):
    T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
    kerrT = T * ((math.sqrt(1 - spin ** 2)) / (1 + math.sqrt(1 - spin ** 2)))
    return(kerrT)

def radius(mass, spin):
    R = (cst.G * mass)/(cst.c**2)
    kerrR = R * (1 + math.sqrt(1 - spin ** 2))
    return(kerrR)
```

```
def SA(r):
    SA = 4 * cst.pi * (r ** 2)
    return SA

def energyloss(temp, SA):
    P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
    return(P)

def simenergyloss(mass, spin):
    e = energyloss(temp(mass, spin), SA(radius(mass, spin)))
    return(e)

def energymass(energy):
    m = energy/(cst.c ** 2)
    return(m)

def angmomspin(J, mass):
    s = (cst.c * J) / (cst.G * mass ** 2)
    return(s)

def spinloss(mass, spin):
    J = energymass(simenergyloss(mass, spin)) * ((spin * cst.G * mass) / cst.c)
    s = angmomspin(J, mass)
    return(s)

def evaptime(mass, spin, ts):

    newm = mass
    news = spin
    time = 0

    while newm > 1:
        e = simenergyloss(newm, news)
        s = spinloss(newm, news)
        tempdata.append(temp(newm, news))

        newm = newm - ((energymass(e) * ts) * max(random.gauss(1.0, noise), 0))
        news = news - ((s * ts) * max(random.gauss(1.0, noise), 0))

        if time == addmasstime: newm += addmass
        time += ts

        massdata.append(newm)
        spindata.append(news)
        raddata.append(radius(newm, news))
    return(time)

resetdata()
et = evaptime(blackholemass, blackholespin, timestep)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et /
86400) + " days \n ~" + str(et / 31557945.6) + " years")
```

```
def timescale(time):
    if time <= 60: return(0)
    elif time <= 3600: return(1)
    elif time <= 86400: return(2)
    elif time <= 31557945.6: return(3)
    else: return(4)

def time(array):
    time = []
    for i in range(len(array)):
        if timescale(et) == 0: time.append(i * timestep)
        elif timescale(et) == 1: time.append(i * timestep/60)
        elif timescale(et) == 2: time.append(i * timestep/3600)
        elif timescale(et) == 3: time.append(i * timestep/86400)
        else: time.append(i * timestep/31557945.6)
    return(time)

def timeunits(time):
    if timescale(time) == 0: return('seconds')
    elif timescale(time) == 1: return('minutes')
    elif timescale(time) == 2: return('hours')
    elif timescale(time) == 3: return('days')
    else: return('years')

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass/Spin Over Time')

ax1.plot(time(massdata), massdata, color='black')
ax1.invert_yaxis
ax1.set_ylim(0, blackholemass*1.1)
ax1.set_xlabel('Time (' + timeunits(et) + ')')
ax1.set_ylabel('Mass (kg)')

ax2 = ax1.twinx()
ax2.plot(time(spindata), spindata, color='red')
ax2.set_ylim(0, 1)
ax2.set_ylabel('Spin (dimensionless)', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Radius/Temp Over Time')

raddata = raddata[1:]
ax3.plot(time(raddata), raddata, color='black')
ax3.set_ylim(0, max(raddata)*1.1)
ax3.set_xlabel('Time (' + timeunits(et) + ')')
ax3.set_ylabel('Radius (m)')

tempdata = tempdata[2:]
ax4 = ax3.twinx()
ax4.plot(time(tempdata), tempdata, color='red')
ax4.set_ylim(tempdata[0], tempdata[-1])
ax4.set_yscale('log')
```

```
ax4.set_ylabel('Temp (K)', color="red")

# Initial Conditions Predictor

truemassdata = massdata.copy()

alldata = []
accuracydata = []
precision = predprecision

def predict(data):
    alldata.clear()
    accuracydata.clear()
    resetdata()

    truedata = data.copy()
    tmass = massmin
    tdata = []

    while tmass <= massmax: # Generates an array of predicted curves in a set range with a set precision
        resetdata()
        ts = (tmass**3)/precision
        evaptime(tmass, 0, ts)
        alldata.append(massdata.copy())
        tmass *= 10/grain

    for i in alldata: # Calculates the difference between each item in the predicted curve and the original and saves it to
accuracydata
        tdata.clear()
        for loc, j in enumerate(i):
            tdata.append(abs(truedata[loc] - j) / abs(truedata[loc]))
        accuracydata.append(tdata.copy()[1:])

def bestmatch(data):

    avgdata = []
    bestmatchloc = 0

    for i in data: # Takes the average difference of each point on a curve so each curve is represented by 1 accuracy value
        avg = 0
        for j in i:
            avg += j
        avgdata.append(avg/len(i))

    bestmatch = avgdata[0]

    for loc, i in enumerate(avgdata): # Sequentially finds the curve with the lowest difference and displays it as the new closest
match
        if i < bestmatch:
            bestmatch = i
            bestmatchloc = loc - 1
            print("New best match: " + f"{massmin*(10/grain)**bestmatchloc:.2e}" + "kg")
        loc+=1
    return(bestmatch, bestmatchloc)
```

```
grain = 9.5 # Larger = more accuracy, DNE > 10 (log = 10/grain)
massmin = 1e34 # Smallest mass scanned
massmax = 1e37 # Largest mass scanned

inputdata = truemassdata

predict(inputdata)
best = bestmatch(accuracydata.copy())
closestmatch = ((10/grain)**(best[1]))*massmin
print('Closest match: ~' + f"{closestmatch:e}" + 'kg\nAccuracy: ' + str(100-(abs(100-round((closestmatch/blackholemass)*100))))
+ '%')

massdata.clear()
pet = evaptime(closestmatch, 0, (closestmatch**3)/precision)
predicted = massdata
real = truemassdata

fig, ax5 = plt.subplots()

plt.title('Black Hole Mass (Predicted vs Real) Over Time [' + str(100-(abs(100-round((closestmatch/blackholemass)*100)))) + '%
Accurate]')
precision = predprecision
timestep = (closestmatch**3)/predprecision
ax5.plot(time(predicted), predicted, color='black')
ax5.set_ylabel('Predicted (kg)', color="black")
ax5.set_ylim(0, closestmatch)
ax5.set_xlabel('Time (' + timeunits(pet) + ')')
ax5.set_xlim(0)

ax6 = ax5.twinx()
timestep = (blackholemass**3)/realprecision
ax6.plot(time(real), real, color='red')
ax6.set_ylabel('Real (kg)', color="red")
ax6.set_ylim(0, blackholemass)
```

Version 12 Notes
Feb. 23, 2026

- I primarily cleaned things up and added more comments in this version
- I changed **tempdata** to **tdata** and **tempmass** to **tmass**
    - **tempdata** and **tempmass** store *temporary* values; however, the simulator model also uses an array **tempdata,** which stores *temperature* data. If I change the prediction model to be based on temperature data rather than mass data, that will cause several issues with conflicting arrays
- I also based the **predict()** function on the input **inputdata** which can be changed down the line
- Added more comments to improve the readability of the prediction model
- Implemented the **enumerate()** function to replace the **loc = 0; for i in x: loc += 1** system to keep track of array item location, which is cleaner and more reliable
- I added a graphing function that plots the real mass curve vs the best match predicted mass curve

Bugs/Issues

- Precision has been greatly reduced in favor of computability; however, infinite precision is still theoretically possible using the simulator
- The simulator still uses standard 64-bit float variables, which can store data with limited precision
- The prediction algorithm does not include spin
- There are likely still many small bugs and edge case scenarios that need to be fixed in the prediction algorithm
- It also uses mass data instead of temperature data for simplicity; however, temperature data is much more realistic for real-world measurements

Version 13 Code
Feb. 23, 2026

```
# Feb. 23, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
import math
import random

massdata = []
tempdata = []
raddata = []
spindata = []

def resetdata():
    massdata.clear()
    tempdata.clear()
    raddata.clear()
    spindata.clear()
resetdata()

# Initial conditions

kg = 8.6e36 # Mass in kg
solarmass = 0 # Mass in M⊙ (1 M⊙ = mass of our Sun)
blackholemass = kg + solarmass * 1.988416e30
blackholespin = 0 # Value must be between 0 > x > 1, 0 = no rotation
noise = 0.5 # Gaussian distribution

# Higher number = more accurate, longer compute time
realprecision = 1e20 # ~1e21 for my pc
predprecision = 1e18 # ~1e19 for my pc

precision = realprecision
timestep = 0

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

timestep = (blackholemass**3)/precision #seconds

def temp(mass, spin):
    T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
    kerrT = T * ((math.sqrt(1 - spin ** 2)) / (1 + math.sqrt(1 - spin ** 2)))
    return(kerrT)

def radius(mass, spin):
    R = (cst.G * mass)/(cst.c**2)
    kerrR = R * (1 + math.sqrt(1 - spin ** 2))
    return(kerrR)
```

```
def SA(r):
    SA = 4 * cst.pi * (r ** 2)
    return SA

def energyloss(temp, SA):
    P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
    return(P)

def simenergyloss(mass, spin):
    e = energyloss(temp(mass, spin), SA(radius(mass, spin)))
    return(e)

def energymass(energy):
    m = energy/(cst.c ** 2)
    return(m)

def angmomspin(J, mass):
    s = (cst.c * J) / (cst.G * mass ** 2)
    return(s)

def spinloss(mass, spin):
    J = energymass(simenergyloss(mass, spin)) * ((spin * cst.G * mass) / cst.c)
    s = angmomspin(J, mass)
    return(s)

def evaptime(mass, spin, ts):

    newm = mass
    news = spin
    time = 0

    while newm > 1:
        e = simenergyloss(newm, news)
        s = spinloss(newm, news)
        tempdata.append(temp(newm, news))

        newm = newm - ((energymass(e) * ts) * max(random.gauss(1.0, noise), 0))
        news = news - ((s * ts) * max(random.gauss(1.0, noise), 0))

        if time == addmasstime: newm += addmass
        time += ts

        massdata.append(newm)
        spindata.append(news)
        raddata.append(radius(newm, news))
    return(time)

resetdata()
et = evaptime(blackholemass, blackholespin, timestep)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et /
86400) + " days \n ~" + str(et / 31557945.6) + " years")
```

```
def timescale(time):
    if time <= 60: return(0)
    elif time <= 3600: return(1)
    elif time <= 86400: return(2)
    elif time <= 31557945.6: return(3)
    else: return(4)

def time(array):
    time = []
    for i in range(len(array)):
        if timescale(et) == 0: time.append(i * timestep)
        elif timescale(et) == 1: time.append(i * timestep/60)
        elif timescale(et) == 2: time.append(i * timestep/3600)
        elif timescale(et) == 3: time.append(i * timestep/86400)
        else: time.append(i * timestep/31557945.6)
    return(time)

def timeunits(time):
    if timescale(time) == 0: return('seconds')
    elif timescale(time) == 1: return('minutes')
    elif timescale(time) == 2: return('hours')
    elif timescale(time) == 3: return('days')
    else: return('years')

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass/Spin Over Time')

ax1.plot(time(massdata), massdata, color='black')
ax1.invert_yaxis
ax1.set_ylim(0, blackholemass*1.1)
ax1.set_xlabel('Time (' + timeunits(et) + ')')
ax1.set_ylabel('Mass (kg)')

ax2 = ax1.twinx()
ax2.plot(time(spindata), spindata, color='red')
ax2.set_ylim(0, 1)
ax2.set_ylabel('Spin (dimensionless)', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Radius/Temp Over Time')

raddata = raddata[1:]
ax3.plot(time(raddata), raddata, color='black')
ax3.set_ylim(0, max(raddata)*1.1)
ax3.set_xlabel('Time (' + timeunits(et) + ')')
ax3.set_ylabel('Radius (m)')

tempdata = tempdata[2:]
ax4 = ax3.twinx()
ax4.plot(time(tempdata), tempdata, color='red')
ax4.set_ylim(tempdata[0], tempdata[-1])
ax4.set_yscale('log')
```

```
ax4.set_ylabel('Temp (K)', color="red")

# Initial Conditions Predictor

truemassdata = massdata.copy()

alldata = []
accuracydata = []
precision = predprecision

def predict(data):
    alldata.clear()
    accuracydata.clear()
    resetdata()

    spinmin = 0
    spinmax = 1

    truedata = data.copy()
    tmass = massmin
    tspin = spinmin
    tdata = []

    while tspin <= spinmax:
        while tmass <= massmax: # Generates an array of predicted curves in a set range with a set precision
            resetdata()
            ts = (tmass**3)/precision
            evaptime(tmass, tspin, ts)
            alldata.append(massdata.copy())
            tmass *= 10/grain
        tspin += 0.1

    for i in alldata: # Calculates the difference between each item in the predicted curve and the original and saves it to
accuracydata
        tdata.clear()
        for loc, j in enumerate(i):
            tdata.append(abs(truedata[loc] - j) / abs(truedata[loc]))
        accuracydata.append(tdata.copy()[1:])

def bestmatch(data):

    avgdata = []
    bestmatchloc = 0

    for i in data: # Takes the average difference of each point on a curve so each curve is represented by 1 accuracy value
        avg = 0
        for j in i:
            avg += j
        avgdata.append(avg/len(i))

    bestmatch = avgdata[0]

    for loc, i in enumerate(avgdata): # Sequentially finds the curve with the lowest difference and displays it as the new closest
match
```

```
        if i < bestmatch:
            bestmatch = i
            bestmatchloc = loc - 1
            print("New best mass: ", f"{massmin*(10/grain)**(bestmatchloc % len(avgdata)/10):.2e}", "kg")
            print("New best spin: ", 10*(bestmatchloc / len(avgdata)/10), '\n')
            print(loc)
        loc += 1

    return(bestmatch, bestmatchloc, len(avgdata))

grain = 9.5 # Larger = more accuracy, DNE > 10 (log = 10/grain)
massmin = 1e30 # Smallest mass scanned
massmax = 1e40 # Largest mass scanned

inputdata = truemassdata

var = len(inputdata)

predict(inputdata)
best = bestmatch(accuracydata.copy())
closestmass = ((10/grain)**(best[1]))*massmin
closestspin = best[1] / best[2]
predaccuracy = closestmass/blackholemass
print('Predicted Mass: ~' + f"{closestmass:e}" + 'kg')
print("Predicted Spin: " + str(closestspin))
print('Accuracy: ' + str(predaccuracy))

massdata.clear()
timestep = (closestmass**3)/predprecision
pet = evaptime(closestmass, closestspin, (closestmass**3)/precision)
predicted = massdata
real = truemassdata

fig, ax5 = plt.subplots()

plt.title('Black Hole Mass (Predicted vs Real) Over Time [' + f"{predaccuracy:.5f}" + ']')
timestep = (closestmass**3)/predprecision
ax5.plot(time(predicted), predicted, color='black')
ax5.set_ylabel('Predicted (kg)', color="black")
ax5.set_ylim(0, closestmass)
ax5.set_xlabel('Time (' + timeunits(pet) + ')')
ax5.set_xlim(0)

ax6 = ax5.twinx()
timestep = (blackholemass**3)/realprecision
ax6.plot(time(real), real, color='red')
ax6.set_ylabel('Real (kg)', color="red")
ax6.set_ylim(0, blackholemass)
```

Version 13 Notes
Feb. 23, 2026

- In version 13, I attempted (and failed) to implement a basic spin prediction parameter
    - I planned to utilize the same prediction model, using inverse modeling and estimation, however run it an additional 10 times to test for spins in range [0.0, 1.0] (it's a very redundant system as it would only pick a new best curve if there were a more accurate curve than any @ spin = 0, so the main function would be unaffected even if the spin prediction were inaccurate and messy)
    - However, I kept running into bugs and issues implementing it and getting an accurate prediction, so I'm scrapping the idea for now, I potentially will come back to it later but I am reverting to version 12 for now
        - If attempting it again, I should probably use a 2-dimensional grid array rather than stacking them all in a 1-dimensional array, since using a 1-dimensional array for a 2-dimensional model is risky and very sensitive (could also be the reason I was experiencing so many issues)
    - My program is getting increasingly complex, as it now requires storing 4 degrees of freedom (which is making it much more difficult to program and debug)
    - The hierarchy of my data functions as such:
        - Total data array (stores everything for easy access)
            - Layer 1 (curves of differing spin)
                - Layer 2 (curves of differing mass)
                    - Individual data points on each curve

Version 14 Code
Feb. 24, 2026

```
# Feb. 24, 2026

import scipy.constants as cst
import matplotlib.pyplot as plt
import numpy as np
import math
import random

massdata = []
tempdata = []
raddata = []
spindata = []

def resetdata():
    massdata.clear()
    tempdata.clear()
    raddata.clear()
    spindata.clear()
resetdata()

# Initial conditions

kg = 8.6e36 # Mass in kg
solarmass = 0 # Mass in M⊙ (1 M⊙ = mass of our Sun)
blackholemass = kg + solarmass * 1.988416e30
blackholespin = 0 # Value must be between 0 > x > 1, 0 = no rotation
noise = 0 # Gaussian distribution

# Higher number = more accurate, longer compute time
realprecision = 1e21 # ~1e21 for my pc
predprecision = 1e17 # ~1e19 for my pc

precision = realprecision
timestep = 0

# Add mass
addmass = 0 #kg
addmasstime = 0 #seconds

timestep = (blackholemass**3)/precision #seconds

def temp(mass, spin):
    T = (cst.hbar * (cst.c ** 3))/(8 * cst.pi * cst.G * mass * cst.k)
    kerrT = T * ((math.sqrt(1 - spin ** 2)) / (1 + math.sqrt(1 - spin ** 2)))
    return(kerrT)

def radius(mass, spin):
    R = (cst.G * mass)/(cst.c**2)
    kerrR = R * (1 + math.sqrt(1 - spin ** 2))
    return(kerrR)
```

```
def SA(r):
    SA = 4 * cst.pi * (r ** 2)
    return SA

def energyloss(temp, SA):
    P = (SA * cst.Stefan_Boltzmann * (temp ** 4))
    return(P)

def simenergyloss(mass, spin):
    e = energyloss(temp(mass, spin), SA(radius(mass, spin)))
    return(e)

def energymass(energy):
    m = energy/(cst.c ** 2)
    return(m)

def angmomspin(J, mass):
    s = (cst.c * J) / (cst.G * mass ** 2)
    return(s)

def spinloss(mass, spin):
    J = energymass(simenergyloss(mass, spin)) * ((spin * cst.G * mass) / cst.c)
    s = angmomspin(J, mass)
    return(s)

def evaptime(mass, spin, ts):

    newm = mass
    news = spin
    time = 0

    while newm > 1:
        e = simenergyloss(newm, news)
        s = spinloss(newm, news)
        tempdata.append(temp(newm, news))

        newm = newm - ((energymass(e) * ts) * max(random.gauss(1.0, noise), 0))
        news = news - ((s * ts) * max(random.gauss(1.0, noise), 0))

        if time == addmasstime: newm += addmass
        time += ts

        massdata.append(newm)
        spindata.append(news)
        raddata.append(radius(newm, news))
    return(time)

resetdata()
et = evaptime(blackholemass, blackholespin, timestep)

print("Evaporated in \n ~" + str(et) + " seconds \n ~" + str(et / 60) + " minutes \n ~" + str(et / 3600) + " hours \n ~" + str(et /
86400) + " days \n ~" + str(et / 31557945.6) + " years")
```

```
def timescale(time):
    if time <= 60: return(0)
    elif time <= 3600: return(1)
    elif time <= 86400: return(2)
    elif time <= 31557945.6: return(3)
    else: return(4)

def time(array):
    time = []
    for i in range(len(array)):
        if timescale(et) == 0: time.append(i * timestep)
        elif timescale(et) == 1: time.append(i * timestep/60)
        elif timescale(et) == 2: time.append(i * timestep/3600)
        elif timescale(et) == 3: time.append(i * timestep/86400)
        else: time.append(i * timestep/31557945.6)
    return(time)

def timeunits(time):
    if timescale(time) == 0: return('seconds')
    elif timescale(time) == 1: return('minutes')
    elif timescale(time) == 2: return('hours')
    elif timescale(time) == 3: return('days')
    else: return('years')

fig, ax1 = plt.subplots()

plt.title('Black Hole Mass/Spin Over Time')

ax1.plot(time(massdata), massdata, color='black')
ax1.invert_yaxis
ax1.set_ylim(0, blackholemass*1.1)
ax1.set_xlabel('Time (' + timeunits(et) + ')')
ax1.set_ylabel('Mass (kg)')

ax2 = ax1.twinx()
ax2.plot(time(spindata), spindata, color='red')
ax2.set_ylim(0, 1)
ax2.set_ylabel('Spin (dimensionless)', color="red")

fig, ax3 = plt.subplots()

plt.title('Black Hole Radius/Temp Over Time')

raddata = raddata[1:]
ax3.plot(time(raddata), raddata, color='black')
ax3.set_ylim(0, max(raddata)*1.1)
ax3.set_xlabel('Time (' + timeunits(et) + ')')
ax3.set_ylabel('Radius (m)')

tempdata = tempdata[2:]
ax4 = ax3.twinx()
ax4.plot(time(tempdata), tempdata, color='red')
ax4.set_ylim(tempdata[0], tempdata[-1])
ax4.set_yscale('log')
```

```
ax4.set_ylabel('Temp (K)', color="red")

# Initial Conditions Predictor

truemassdata = massdata.copy()

alldataL1 = []
accuracydataL1 = []
accuracydataL2 = []
precision = predprecision

def predict(data):
    alldataL1.clear()
    accuracydataL1.clear()
    accuracydataL2.clear()
    resetdata()

    alldataL2 = []

    truedata = data.copy()
    tspin = spinmin
    tmass = massmin
    tdata = []

    print('1')

    while tspin <= spinmax:
        alldataL2 = []
        print(tspin)
        tmass = massmin
        while tmass <= massmax: # Generates an array of predicted curves in a set range with a set precision
            resetdata()
            ts = (tmass**3)/precision
            evaptime(tmass, tspin, ts)
            alldataL2.append(massdata.copy())
            tmass *= 10/grain
        alldataL1.append([curve.copy() for curve in alldataL2])
        tspin += 0.1

    print('2')

    for i in alldataL1:
        accuracydataL2 = []
        for j in i: # Calculates the difference between each item in the predicted curve and the original and saves it to accuracydata
            tdata = []
            for loc, k in enumerate(j):
                tdata.append(abs(truedata[loc] - k) / abs(truedata[loc]))
            accuracydataL2.append(tdata.copy()[1:])
        accuracydataL1.append(accuracydataL1.copy())

def bestmatch(data):

    avgdataL1 = []
    avgdataL2 = []
```

```
    bestmatchloc = 0

    print('3')

    for i in data:
        for j in i: # Takes the average difference of each point on a curve so each curve is represented by 1 accuracy value
            avg = 0
            avgdataL2 = []
            for k in j:
                avg += k
            if len(j) > 0: avgdataL2.append(avg/len(j))
            avgdataL1.append(avgdataL2.copy())

    print('4')

    bestmatch = avgdataL1[0][1]
    for loc1, i in enumerate(avgdataL1):
        for loc2, j in enumerate(avgdataL2): # Sequentially finds the curve with the lowest difference and displays it as the new
closest match
            if j < bestmatch:
                bestmatch = j
                bestmatchloc = loc - 1
                print("New best match: " + f"{massmin*(10/grain)**bestmatchloc:.2e}" + "kg")
                print(loc1)
            loc2+=1
    return(bestmatch, bestmatchloc)

grain = 5 # Larger = more accuracy, DNE > 10 (log = 10/grain)
massmin = 1e10 # Smallest mass scanned
massmax = 1e12 # Largest mass scanned
spinmin = 0
spinmax = 0.4
predprecision = 1e19
precision = predprecision

inputdata = truemassdata

predict(inputdata)
accuracydataL1s
best = bestmatch(accuracydataL1.copy())
closestmatch = ((10/grain)**(best[1]))*massmin
print('Closest match: ~' + f"{closestmatch:e}" + 'kg\nAccuracy: ' + str(100-(abs(100-round((closestmatch/blackholemass)*100))))
+ '%')

massdata.clear()
pet = evaptime(closestmatch, 0, (closestmatch**3)/precision)
predicted = massdata
real = truemassdata

fig, ax5 = plt.subplots()

plt.title('Black Hole Mass (Predicted vs Real) Over Time [' + str(100-(abs(100-round((closestmatch/blackholemass)*100)))) + '%
Accurate]')
precision = predprecision
```

```
timestep = (closestmatch**3)/predprecision
ax5.plot(time(predicted), predicted, color='black')
ax5.set_ylabel('Predicted (kg)', color="black")
ax5.set_ylim(0, closestmatch)
ax5.set_xlabel('Time (' + timeunits(pet) + ')')
ax5.set_xlim(0)

ax6 = ax5.twinx()
timestep = (blackholemass**3)/realprecision
ax6.plot(time(real), real, color='red')
ax6.set_ylabel('Real (kg)', color="red")
ax6.set_ylim(0, blackholemass)
```

Version 14 Notes
Feb. 24, 2026

- I attempted the spin prediction model again utilizing a different method and starting from scratch off version 12, however I still ran into countless bugs and errors and have determined it to be a fundamentally structural issue
    - My new system using 2 layers of data storage, **alldataL1** and **alldataL2,** the latter storing each generated data curve for each mass in the range, and the former storing the **alldataL2** arrays generated with each spin within the range
    - While still hardly a 2 dimensional system, using 1 dimensional spliced arrays it's (in theory) much simpler and easier for debugging as I can read each data curve as **i in alldataL1: j in i**
    - However I ran into several bugs and issues constantly, partially caused by the implementation of the layer system, caused by varying factors such as:
        - **Division by zero error**
        - **Empty array generation/indexing**
        - **Incorrect array appending**
        - **Issues with spin calculation above ~0.4**
        - **Overflow errors**
        - **clear() function issues**
        - **Infinite/extremely long run loops**
        - **Local/global variable mismatches**
- In the future, I will likely attempt to rewrite the prediction algorithm using **numpy** arrays which can store much cleaner grids and supersets, however, in order to implement that I would need to rewrite the entire system using said arrays which is out of scope and time is limited (for regionals)

Bugs/Issues

- Precision has been greatly reduced in favor of computability; however, infinite precision is still theoretically possible using the simulator
- The simulator still uses standard 64-bit float variables, which can store data with limited precision
- The prediction algorithm does not include spin (correctly)
- There are likely still many small bugs and edge case scenarios that need to be fixed in the prediction algorithm
- It also uses mass data instead of temperature data for simplicity; however, temperature data is much more realistic for real-world measurements

Generated Data

The following are examples of generated data with set constraints/inputs (using version 12)

DATA 1

- **Simulator inputs**
    - **Initial mass = 8.36e36kg**
    - **Spin = 0**
    - **Soise = 0**
    - **Precision = 1e21**
    - **Add Mass = 0**
    - **Add Mass Time = 0**
- **Prediction model inputs**
    - **Grain = 9**
    - **Mass Min = 1e34kg**
    - **Mass Max = 1e38kg**
    - **Precision = 1e19**

Mass Data (First 100, out of 1,345,843)

8.599997869973995e+36kg, 8.599995739946935e+36kg, 8.59999360991882e+36kg, 8.59999147988965e+36kg, 8.599989349859424e+36kg, 8.599987219828144e+36kg, 8.599985089795808e+36kg, 8.599982959762417e+36kg, 8.59998082972797e+36kg, 8.5999769969247e+36kg, 8.599976569655913e+36kg, 8.599974439618302e+36kg, 8.599972309579636e+36kg, 8.599970179539914e+36kg, 8.599968049499137e+36kg, 8.599965919457305e+36kg, 8.599963789414418e+36kg, 8.599961659370476e+36kg, 8.599959529325479e+36kg, 8.599957399279426e+36kg, 8.599955269232319e+36kg, 8.599953139184156e+36kg, 8.599951009134938e+36kg, 8.599948879084665e+36kg, 8.599946749033337e+36kg, 8.599944618980953e+36kg, 8.599942488927515e+36kg, 8.599940358873021e+36kg, 8.599938228817472e+36kg, 8.599936098760868e+36kg, 8.599933968703209e+36kg, 8.599931838644494e+36kg, 8.599929708584725e+36kg, 8.599927578523901e+36kg, 8.599925448462021e+36kg, 8.599923318399085e+36kg, 8.599921188335096e+36kg, 8.59991905827005e+36kg, 8.59991692820395e+36kg, 8.599914798136794e+36kg, 8.599912668068584e+36kg, 8.599910537999318e+36kg, 8.599908407928997e+36kg, 8.59990627785762e+36kg, 8.599904147785189e+36kg, 8.599902017711703e+36kg, 8.599899887637161e+36kg, 8.599897757561564e+36kg, 8.599895627484911e+36kg, 8.599893497407204e+36kg, 8.599891367328442e+36kg, 8.599889237248624e+36kg, 8.599887107167751e+36kg, 8.599884977085823e+36kg, 8.59988284700284e+36kg, 8.599880716918802e+36kg, 8.599878586833708e+36kg, 8.599876456747558e+36kg, 8.599874326660355e+36kg, 8.599872196572096e+36kg, 8.599870066482781e+36kg, 8.599867936392411e+36kg, 8.599865806300986e+36kg, 8.599863676208506e+36kg, 8.599861546114971e+36kg, 8.59985941602038e+36kg, 8.599857285924734e+36kg, 8.599855155828034e+36kg, 8.599853025730278e+36kg, 8.599850895631467e+36kg, 8.5998487655316e+36kg, 8.599846635430678e+36kg, 8.599844505328702e+36kg, 8.59984237522567e+36kg, 8.599840245121583e+36kg, 8.59983811501644e+36kg, 8.599835984910242e+36kg, 8.59983385480299e+36kg, 8.599831724694682e+36kg, 8.599829594585318e+36kg, 8.5998274644749e+36kg, 8.599825334363425e+36kg, 8.599823204250897e+36kg, 8.599821074137313e+36kg, 8.599818944022673e+36kg, 8.599816813906979e+36kg, 8.599814683790228e+36kg, 8.599812553672423e+36kg, 8.599810423553563e+36kg, 8.599808293433647e+36kg, 8.599806163312676e+36kg, 8.59980403319065e+36kg, 8.599801903067568e+36kg, 8.599799772943432e+36kg, 8.59979764281824e+36kg, 8.599795512691993e+36kg, 8.599793382564691e+36kg, 8.599791252436333e+36kg, 8.59978912230692e+36kg, 8.599786992176452e+36kg, 8.599784862044929e+36kg

Spin Data (First 100, out of 1,345,843)

0.0, 0.0, 0.0,

Temperature Data (First 100, out of 1,345,843)

7.1331469671262l6e-15K, 7.133148733848876e-15K, 7.133150500573287e-15K, 7.133152267299449e-15K,

7.13315403402736e-15K, 7.133155800757021e-15K, 7.133157567488432e-15K, 7.133159334221596e-15K,

7.133161100956507e-15K, 7.133162867693172e-15K, 7.133164634431586e-15K, 7.133166401171749e-15K,

7.133168167913663e-15K, 7.133169934657327e-15K, 7.133171701402742e-15K, 7.133173468149907e-15K,

7.13317523489882e-15K, 7.133177001649487e-15K, 7.133178768401904e-15K, 7.13318053515607e-15K,

7.133182301911986e-15K, 7.133184068669654e-15K, 7.133185835429071e-15K, 7.13318760219024e-15K,

7.133189368953158e-15K, 7.133191135717828e-15K, 7.133192902484246e-15K, 7.133194669252416e-15K,

7.133196436022335e-15K, 7.133198202794005e-15K, 7.133199969567427e-15K, 7.133201736342598e-15K,

7.13320350311952e-15K, 7.133205269898192e-15K, 7.133207036678615e-15K, 7.133208803460788e-15K,

7.133210570244709e-15K, 7.133212337030385e-15K, 7.133214103817808e-15K, 7.133215870606981e-15K,

7.133217637397906e-15K, 7.133219404190582e-15K, 7.133221170985007e-15K, 7.133222937781183e-15K,

7.133224704579109e-15K, 7.133226471378787e-15K, 7.133228238180214e-15K, 7.133230004983393e-15K,

7.133231771788321e-15K, 7.133233538595e-15K, 7.13323530540343e-15K, 7.13323707221361e-15K,

7.13323883902554e-15K, 7.13324060583922e-15K, 7.133242372654651e-15K, 7.133244139471832e-15K,

7.133245906290765e-15K, 7.133247673111447e-15K, 7.133249439933881e-15K, 7.133251206758064e-15K,

7.133252973583999e-15K, 7.133254740411684e-15K, 7.133256507241118e-15K, 7.133258274072304e-15K,

7.13326004090524e-15K, 7.133261807739926e-15K, 7.133263574576363e-15K, 7.133265341414552e-15K,

7.13326710825449e-15K, 7.133268875096177e-15K, 7.133270641939618e-15K, 7.133272408784807e-15K,

7.133274175631747e-15K, 7.133275942480436e-15K, 7.133277709330879e-15K, 7.13327947618307e-15K,

7.133281243037013e-15K, 7.133283009892706e-15K, 7.133284776750149e-15K, 7.133286543609342e-15K,

7.133288310470289e-15K, 7.133290077332982e-15K, 7.133291844197429e-15K, 7.133293611063623e-15K,

7.13329537793157e-15K, 7.133297144801268e-15K, 7.133298911672716e-15K, 7.133300678545916e-15K,

7.133302445420863e-15K, 7.133304212297562e-15K, 7.133305979176013e-15K, 7.133307746056214e-15K,

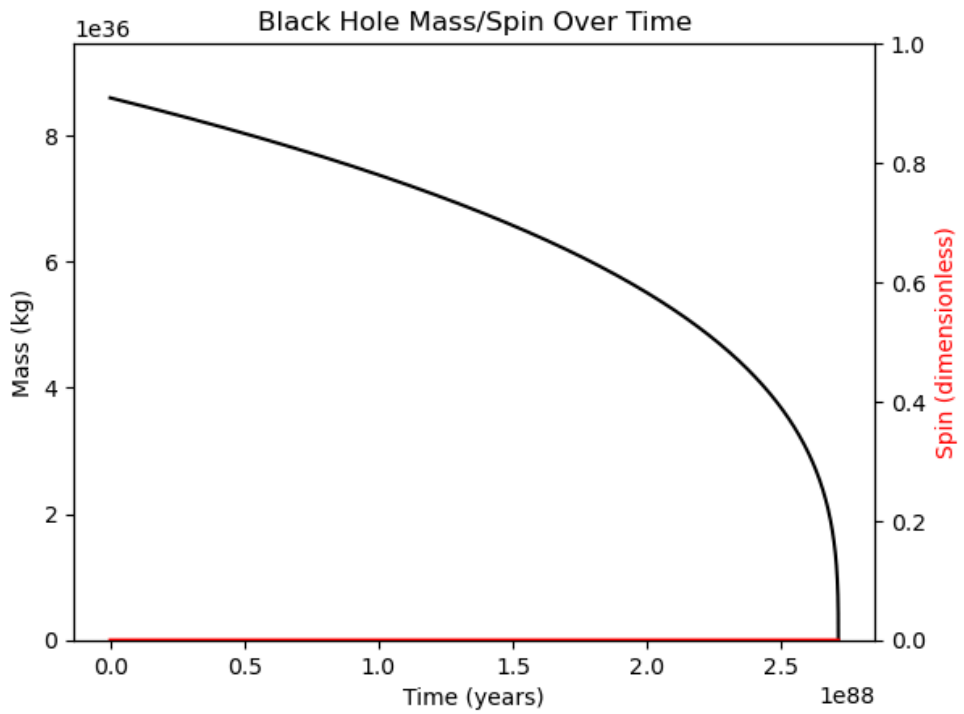
7.133309512938165e-15K, 7.133311279821866e-15K, 7.133313046707318e-15K, 7.133314813594522e-15K,

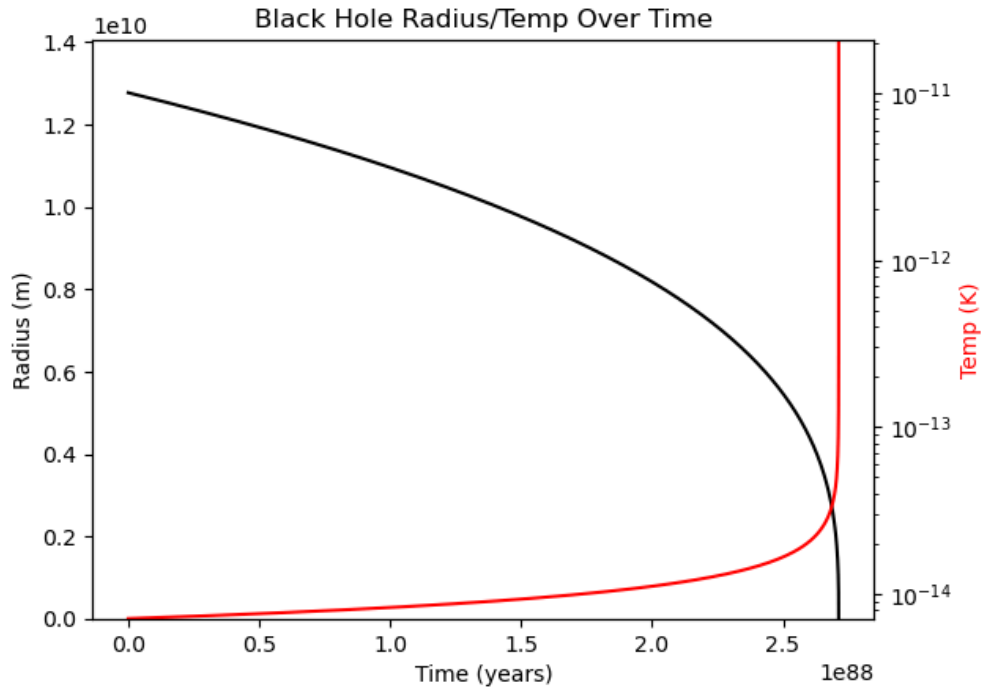
7.133316580483477e-15K, 7.133318347374181e-15K, 7.133320114266635e-15K, 7.13332188116084e-15K,

7.133323648056797e-15K

Radius Data (First 100, out of 1,345,843)

12772989335.716742m, 12772986172.130709m, 12772983008.54311m, 12772979844.953941m, 12772976681.363209m, 12772973517.770908m, 12772970354.17704m, 12772967190.581604m, 12772964026.984604m, 12772960863.386034m, 12772957699.785898m, 12772954536.184196m, 12772951372.580925m, 12772948208.976088m, 12772945045.36968m, 12772941881.761711m, 12772938718.152172m, 12772935554.541065m, 12772932390.92839m, 12772929227.314152m, 12772926063.698345m, 12772922900.080969m, 12772919736.462027m, 12772916572.841518m, 12772913409.219442m, 12772910245.595798m, 12772907081.970589m, 12772903918.343811m, 12772900754.715467m, 12772897591.085554m, 12772894427.454075m, 12772891263.82103m, 12772888100.186415m, 12772884936.550236m, 12772881772.912487m, 12772878609.273174m, 12772875445.632292m, 12772872281.989841m, 12772869118.345825m, 12772865954.700243m, 12772862791.053093m, 12772859627.404375m, 12772856463.75409m, 12772853300.102238m, 12772850136.448818m, 12772846972.793833m, 12772843809.137278m, 12772840645.479156m, 12772837481.81947m, 12772834318.158213m, 12772831154.495392m, 12772827990.831m, 12772824827.165045m, 12772821663.49752m, 12772818499.828428m, 12772815336.157772m, 12772812172.485544m, 12772809008.811752m, 12772805845.136393m, 12772802681.459465m, 12772799517.78097m, 12772796354.100906m, 12772793190.419277m, 12772790026.736082m, 12772786863.05132m, 12772783699.364986m, 12772780535.677092m, 12772777371.987625m, 12772774208.296595m, 12772771044.603994m, 12772767880.909826m, 12772764717.214094m, 12772761553.516792m, 12772758389.817924m, 12772755226.117489m, 12772752062.415485m, 12772748898.711916m, 12772745735.006779m, 12772742571.300074m, 12772739407.5918m, 12772736243.881962m, 12772733080.170557m, 12772729916.457582m, 12772726752.743042m, 12772723589.026934m, 12772720425.309258m, 12772717261.590015m, 12772714097.869205m, 12772710934.146828m, 12772707770.422884m, 12772704606.697372m, 12772701442.970291m, 12772698279.241646m, 12772695115.511435m, 12772691951.779652m, 12772688788.046305m, 12772685624.311388m, 12772682460.574905m, 12772679296.836857m, 12772676133.097239m, 12772672969.356054m





Timestep = 6.36056e+89 seconds per data point

Best Match Data [Prediction Model]

New best match: 1.00e+34kg

New best match: 1.11e+34kg

New best match: 1.23e+34kg

New best match: 1.37e+34kg

New best match: 1.52e+34kg

New best match: 1.69e+34kg

New best match: 1.88e+34kg

New best match: 2.09e+34kg

New best match: 2.32e+34kg

New best match: 2.58e+34kg

New best match: 2.87e+34kg

New best match: 3.19e+34kg

New best match: 3.54e+34kg

New best match: 3.93e+34kg

New best match: 4.37e+34kg

New best match: 4.86e+34kg

New best match: 5.40e+34kg

New best match: 6.00e+34kg

New best match: 6.66e+34kg

New best match: 7.40e+34kg

New best match: 8.23e+34kg

New best match: 9.14e+34kg

New best match: 1.02e+35kg

New best match: 1.13e+35kg

New best match: 1.25e+35kg

New best match: 1.39e+35kg

New best match: 1.55e+35kg

New best match: 1.72e+35kg

New best match: 1.91e+35kg

New best match: 2.12e+35kg

New best match: 2.36e+35kg

New best match: 2.62e+35kg

New best match: 2.91e+35kg

New best match: 3.24e+35kg

New best match: 3.60e+35kg

New best match: 3.99e+35kg

New best match: 4.44e+35kg

New best match: 4.93e+35kg

New best match: 5.48e+35kg

New best match: 6.09e+35kg

New best match: 6.77e+35kg

New best match: 7.52e+35kg

New best match: 8.35e+35kg

New best match: 9.28e+35kg

New best match: 1.03e+36kg

New best match: 1.15e+36kg

New best match: 1.27e+36kg

New best match: 1.41e+36kg

New best match: 1.57e+36kg

New best match: 1.75e+36kg

New best match: 1.94e+36kg

New best match: 2.16e+36kg

New best match: 2.40e+36kg

New best match: 2.66e+36kg

New best match: 2.96e+36kg

New best match: 3.29e+36kg

New best match: 3.65e+36kg

New best match: 4.06e+36kg

New best match: 4.51e+36kg

New best match: 5.01e+36kg

New best match: 5.56e+36kg

New best match: 6.18e+36kg

New best match: 6.87e+36kg

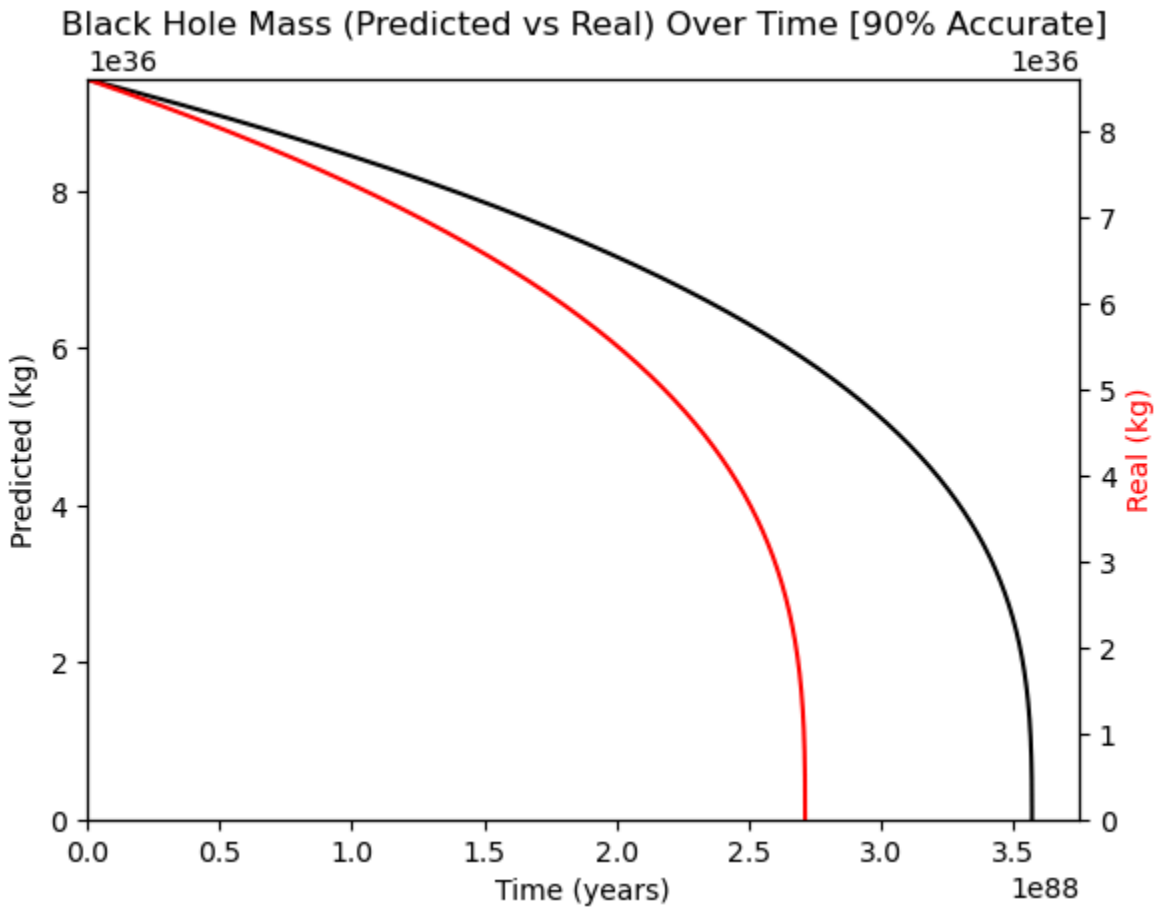
New best match: 7.63e+36kg

New best match: 8.48e+36kg

New best match: 9.42e+36kg

Closest match: ~9.424035e+36kg

Accuracy: 90%



DATA 2

- **Simulator inputs**
    - **Initial mass = 8.6e36kg**
    - **Spin = 0.5**
    - **Noise = 0.5**
    - **Precision = 1e21**
    - **Add Mass = 0**
    - **Add Mass Time = 0**
- **Prediction model inputs**
    - **Grain = 9**
    - **Mass Min = 1e34kg**
    - **Mass Max = 1e38kg**
    - **Precision = 1e19**

Mass Data (First 100, out of 1,345,843)

8.59999808129027e+36kg, 8.59999712770769e+36kg, 8.599994914968261e+36kg, 8.599993822030115e+36kg, 8.599992760872894e+36kg, 8.599991714755851e+36kg, 8.599989680014118e+36kg, 8.599988224416807e+36kg, 8.599987488430228e+36kg, 8.599987088965702e+36kg, 8.599984503903823e+36kg, 8.599983526437519e+36kg, 8.599981650888944e+36kg, 8.59997968857316e+36kg, 8.599979132243208e+36kg, 8.599978007430413e+36kg, 8.599976497428575e+36kg, 8.599974716486455e+36kg, 8.599972975346834e+36kg, 8.599972830270815e+36kg, 8.59997118761432e+36kg, 8.5999705768604e+36kg, 8.59996318546523e+36kg, 8.59996906122258e+36kg, 8.599966994655337e+36kg, 8.59996599684548e+36kg, 8.59996471168832 2e+36kg, 8.599963694782899e+36kg, 8.59996249365387 3e+36kg, 8.59996123325588 5e+36kg, 8.59995972917260 2e+36kg, 8.59995879830996e+36kg, 8.599958421501413e+36kg, 8.59995722097153 9e+36kg, 8.59995599400405 6e+36kg, 8.59995397742135 6e+36kg, 8.599951996156733e+36kg, 8.59994963090114 1e+36kg, 8.59994678227601 3e+36kg, 8.59994494881096 9e+36kg, 8.599943335869166e+36kg, 8.59994252191009e+36kg, 8.599941589198183e+36kg, 8.59993972620947 7e+36kg, 8.599937633391502e+36kg, 8.59993578605612 9e+36kg, 8.59993381384981 5e+36kg, 8.59993201390985 8e+36kg, 8.599931411675149e+36kg, 8.59993008186267 3e+36kg, 8.59993008186267 3e+36kg, 8.59992865125767 9e+36kg, 8.599926847757338e+36kg, 8.59992496197015e+36kg, 8.59992445992417e+36kg, 8.59992386999019 7e+36kg, 8.599922823939582e+36kg, 8.59992135623473 3e+36kg, 8.59991941174022 1e+36kg, 8.59991897868e+36kg, 8.59991682429393e+36kg, 8.59991608659574 4e+36kg, 8.59991494217770 3e+36kg, 8.59991334738809 3e+36kg, 8.599911328598417e+36kg, 8.59991058762793 7e+36kg, 8.59990862963789 3e+36kg, 8.59990784116863 9e+36kg, 8.599906044065943e+36kg, 8.59990530182350 8e+36kg, 8.59990503680527 5e+36kg, 8.59990370309708e+36kg, 8.599902931343016e+36kg, 8.59990139134212 7e+36kg, 8.59989965159429 2e+36kg, 8.59989847305 3e+36kg, 8.599897633993586e+36kg, 8.59989659210653 9e+36kg, 8.59989572517399 6e+36kg, 8.59989424335183 4e+36kg, 8.599893296432145e+36kg, 8.59989185546046 4e+36kg, 8.59989068723824 7e+36kg, 8.59988862573985 7e+36kg, 8.599887016074468e+36kg, 8.59988689537987 9e+36kg, 8.59988577507235 6e+36kg, 8.59988504856992 8e+36kg, 8.599883962153846e+36kg, 8.59988199369565 1e+36kg, 8.59988030239039 2e+36kg, 8.59988006101911 7e+36kg, 8.599879150005924e+36kg, 8.59987791909853e+36kg, 8.59987635220786 1e+36kg, 8.59987536615670 7e+36kg, 8.599872688068501e+36kg, 8.59987089579681e+36kg, 8.59986965514175 5e+36kg, 8.59986780061237 2e+36kg, 8.59986698834836 1e+36kg

Spin Data (First 100, out of 1,345,843)

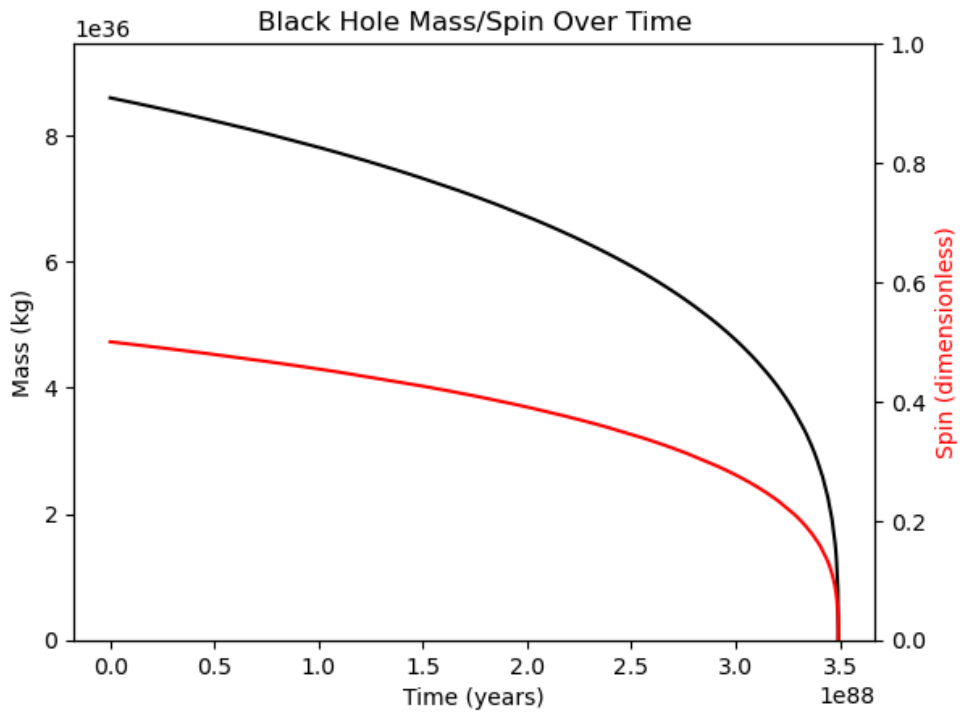
0.4999999162051601, 0.49999984984679285, 0.49999979926266885, 0.49999976253507933, 0.4999997088123364, 0.4999997088123364, 0.499999652960285, 0.4999995812151497, 0.49999952377907897, 0.49999946619405544, 0.4999993933229028, 0.4999992801198127, 0.49999915668781125, 0.49999915668781125, 0.49999900891600907, 0.49999892399897866, 0.4999988723698857, 0.4999987666659765, 0.499998710479827, 0.49999860628979914, 0.49999855516497993, 0.4999984566274854, 0.4999983460024333, 0.4999983138025808, 0.49999821505692865, 0.4999981052858061, 0.4999980360273485, 0.4999978749486033, 0.49999777163085285, 0.4999976982648996, 0.4999975927241056, 0.49999748334999483, 0.4999974594292135, 0.49999732839384886, 0.4999972532010526, 0.49999712249644274, 0.49999705547168316, 0.4999970365567402, 0.4999969557109193, 0.4999969460879524, 0.4999968950754969, 0.49999687187541186, 0.4999967555243851, 0.4999966749132723, 0.49999660433640764, 0.49999656918929414, 0.4999964928237145, 0.49999640527778944, 0.499996330120637, 0.49999621319837867, 0.49999609638481435, 0.4999960459326411, 0.4999959131281725, 0.4999958718578906, 0.49999574369055116, 0.4999957052834837, 0.499995613703008, 0.49999550182463004, 0.4999953852422636, 0.4999953301542419, 0.4999953057934472, 0.4999953057934472, 0.4999952407499357, 0.4999951354775053, 0.4999950257820936, 0.499994962793913, 0.4999949117938614, 0.4999948863708531, 0.4999948492791997, 0.4999947211356889, 0.49999467150573845, 0.4999946128624347, 0.49999451740036593, 0.4999944283195903, 0.49999433861796694, 0.49999429166700954, 0.4999941287005496, 0.4999940566282128, 0.4999939370859592, 0.4999938822769825, 0.4999938822769825, 0.4999938588718901, 0.4999937584625841, 0.49999363803435737, 0.4999935461112109, 0.49999349402681964, 0.49999349402681964, 0.49999337263738275, 0.4999933045117719, 0.49999327619897077, 0.49999311208395836, 0.49999302754886915, 0.4999929443609005, 0.49999285810366406, 0.4999927610613795, 0.4999926466418682, 0.49999253966726254, 0.499992519871445, 0.49999241328298594, 0.4999923268014457, 0.4999922759543697

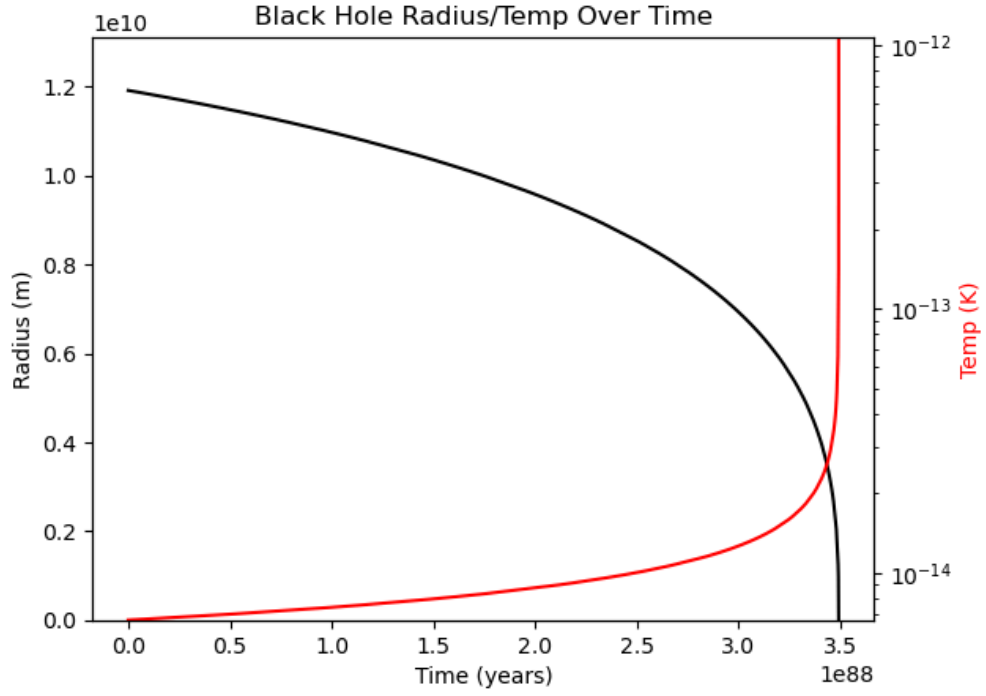
Temperature Data (First 100, out of 1,345,843)

6.621009343681232e-15K, 6.621011166891324e-15K, 6.6210120952062e-15K, 6.6210130392549344e-15K, 6.621013844645776e-15K, 6.621015543281164e-15K, 6.621016833636121e-15K, 6.621017536125076e-15K, 6.621017979882513e-15K, 6.621020142462193e-15K, 6.621021162778393e-15K, 6.621022898713478e-15K, 6.621024409477248e-15K, 6.621025187337013e-15K, 6.621026254185073e-15K, 6.62102753884503e-15K, 6.6210291600110216e-15K, 6.621030633402025e-15K, 6.621030991551377e-15K, 6.621032377149475e-15K, 6.6210330804488315e-15K, 6.6210343108909595e-15K, 6.621034585169465e-15K, 6.621036409778607e-15K, 6.621037437641859e-15K, 6.621038590900688e-15K, 6.621039754831525e-15K, 6.621040923963901e-15K, 6.621042075576651e-15K, 6.6210434755112846e-15K, 6.621044450894421e-15K, 6.621044797579941e-15K, 6.621046031816724e-15K, 6.621047154315137e-15K, 6.6210490160437434e-15K, 6.621050699950545e-15K, 6.6210525656892594e-15K, 6.6210549500670355e-15K, 6.621056384405318e-15K, 6.6210577468691025e-15K, 6.621058428411228e-15K, 6.621059421724597e-15K, 6.621061046713671e-15K, 6.621062824912669e-15K, 6.6210643303089214e-15K, 6.621066029343789e-15K, 6.621067622198455e-15K, 6.621068263637965e-15K, 6.6210695640313595e-15K, 6.6210698403476835e-15K, 6.621071061109875e-15K, 6.621072763764042e-15K, 6.6210743132521715e-15K, 6.621075002949864e-15K, 6.621075547989421e-15K, 6.621076569971791e-15K, 6.621077964599411e-15K, 6.621079737434873e-15K, 6.6210802011556795e-15K, 6.621081917442683e-15K, 6.621082485397074e-15K, 6.621083520342405e-15K, 6.621084997188896e-15K, 6.621086810936541e-15K, 6.621087530406051e-15K, 6.621089158504404e-15K, 6.621089825685419e-15K, 6.621091297017641e-15K, 6.621092171588404e-15K, 6.6210924930235295e-15K, 6.621093658566974e-15K, 6.6210944478552997e-15K, 6.621095874920539e-15K, 6.621097426543073e-15K, 6.6210984449666396e-15K, 6.6210994764496735e-15K, 6.621100449086066e-15K, 6.621101399311555e-15K, 6.621102669821143e-15K, 6.621103398859546e-15K, 6.621104563634804e-15K, 6.62110570056788e-15K, 6.621107572593513e-15K, 6.621109029323719e-15K, 6.621109245449704e-15K, 6.6211101079819155e-15K, 6.621110954460329e-15K, 6.6211119520466e-15K, 6.62111353454912e-15K, 6.6211152249018324e-15K, 6.621115610697903e-15K, 6.621116508870101e-15K, 6.621117660591793e-15K, 6.621119096501507e-15K, 6.621120126322867e-15K, 6.6211224412488245e-15K, 6.6211238679615 83e-15K, 6.621125075282153e-15K, 6.6211267076695126e-15K, 6.621127453315426e-15K, 6.621128216631954e-15K

Radius Data (First 100, out of 1,345,843)

11917363768.085238m, 11917360888.318659m, 11917359509.213545m, 11917358236.81371m, 11917356787.166958m, 11917354173.482786m, 11917352420.943958m, 11917351612.837887m, 11917351271.61324m, 11917347958.08103m, 11917347020.972387m, 11917344877.070694m, 11917342157.80978m, 11917341931.749073m, 11917340686.158432m, 11917338784.055326m, 11917336705.88504m, 11917334500.287476m, 11917334683.4214m, 11917332595.63345m, 11917332112.616022m, 11917330776.817463m, 11917330538.960934m, 11917328039.330233m, 11917327061.373642m, 11917325535.849064m, 11917324720.612562m, 11917323437.1126m, 11917321947.185087m, 11917320265.922722m, 11917319379.272514m, 11917318945.314081m, 11917317764.843956m, 11917316341.83511m, 11917314029.306562m, 11917311530.919239m, 11917308323.029186m, 11917304673.66494m, 11917302168.437746m, 11917300121.409761m, 11917299079.015871m, 11917298215.526402m, 11917295931.133486m, 11917293291.258205m, 11917290860.922224m, 11917288409.527334m, 11917286238.07294m, 11917285680.649096m, 11917284268.985535m, 11917284699.69799m, 11917282903.2746m, 11917280893.761671m, 11917278432.717625m, 11917278209.58646m, 11917277533.703505m, 11917276421.819838m, 11917274800.473686m, 11917272535.764029m, 11917272138.772816m, 11917269243.171005m, 11917268220.911362m, 11917266874.868826m, 11917265053.057709m, 11917262659.999662m, 11917261865.45279m, 11917259340.22792m, 11917258341.351137m, 11917255987.792133m, 11917255431.72009m, 11917255247.466267m, 11917253615.51672m, 11917252898.047255m, 11917251092.456377m, 11917249012.356571m, 11917247552.316538m, 11917246990.477015m, 11917245812.430521m, 11917245051.854212m, 11917243200.517992m, 11917241888.330667m, 11917239977.8121m, 11917238729.179354m, 11917236316.507607m, 11917234424.857931m, 11917234449.648365m, 11917232897.189877m, 11917232338.023031m, 11917231083.71737m, 11917228460.332699m, 11917226721.7309m, 11917226698.943851m, 11917225743.237846m, 11917224355.557186m, 11917222542.055527m, 11917221597.520164m, 11917218280.80486m, 11917215870.16539m, 11917214543.939655m, 11917212292.904734m, 11917211354.79397m, 11917210189.345388m





Timestep = 6.36056e+89 seconds per data point

Best Match Data [Prediction Model]

New best match: 1.00e+34kg

New best match: 1.11e+34kg

New best match: 1.23e+34kg

New best match: 1.37e+34kg

New best match: 1.52e+34kg

New best match: 1.69e+34kg

New best match: 1.88e+34kg

New best match: 2.09e+34kg

New best match: 2.32e+34kg

New best match: 2.58e+34kg

New best match: 2.87e+34kg

New best match: 3.19e+34kg

New best match: 3.54e+34kg

New best match: 3.93e+34kg

New best match: 4.37e+34kg

New best match: 4.86e+34kg

New best match: 5.40e+34kg

New best match: 6.00e+34kg

New best match: 6.66e+34kg

New best match: 7.40e+34kg

New best match: 8.23e+34kg

New best match: 9.14e+34kg

New best match: 1.02e+35kg

New best match: 1.13e+35kg

New best match: 1.25e+35kg

New best match: 1.39e+35kg

New best match: 1.55e+35kg

New best match: 1.72e+35kg

New best match: 1.91e+35kg

New best match: 2.12e+35kg

New best match: 2.36e+35kg

New best match: 2.62e+35kg

New best match: 2.91e+35kg

New best match: 3.24e+35kg

New best match: 3.60e+35kg

New best match: 3.99e+35kg

New best match: 4.44e+35kg

New best match: 4.93e+35kg

New best match: 5.48e+35kg

New best match: 6.09e+35kg

New best match: 6.77e+35kg

New best match: 7.52e+35kg

New best match: 8.35e+35kg

New best match: 9.28e+35kg

New best match: 1.03e+36kg

New best match: 1.15e+36kg

New best match: 1.27e+36kg

New best match: 1.41e+36kg

New best match: 1.57e+36kg

New best match: 1.75e+36kg

New best match: 1.94e+36kg

New best match: 2.16e+36kg

New best match: 2.40e+36kg

New best match: 2.66e+36kg

New best match: 2.96e+36kg

New best match: 3.29e+36kg

New best match: 3.65e+36kg

New best match: 4.06e+36kg

New best match: 4.51e+36kg

New best match: 5.01e+36kg

New best match: 5.56e+36kg

New best match: 6.18e+36kg

New best match: 6.87e+36kg

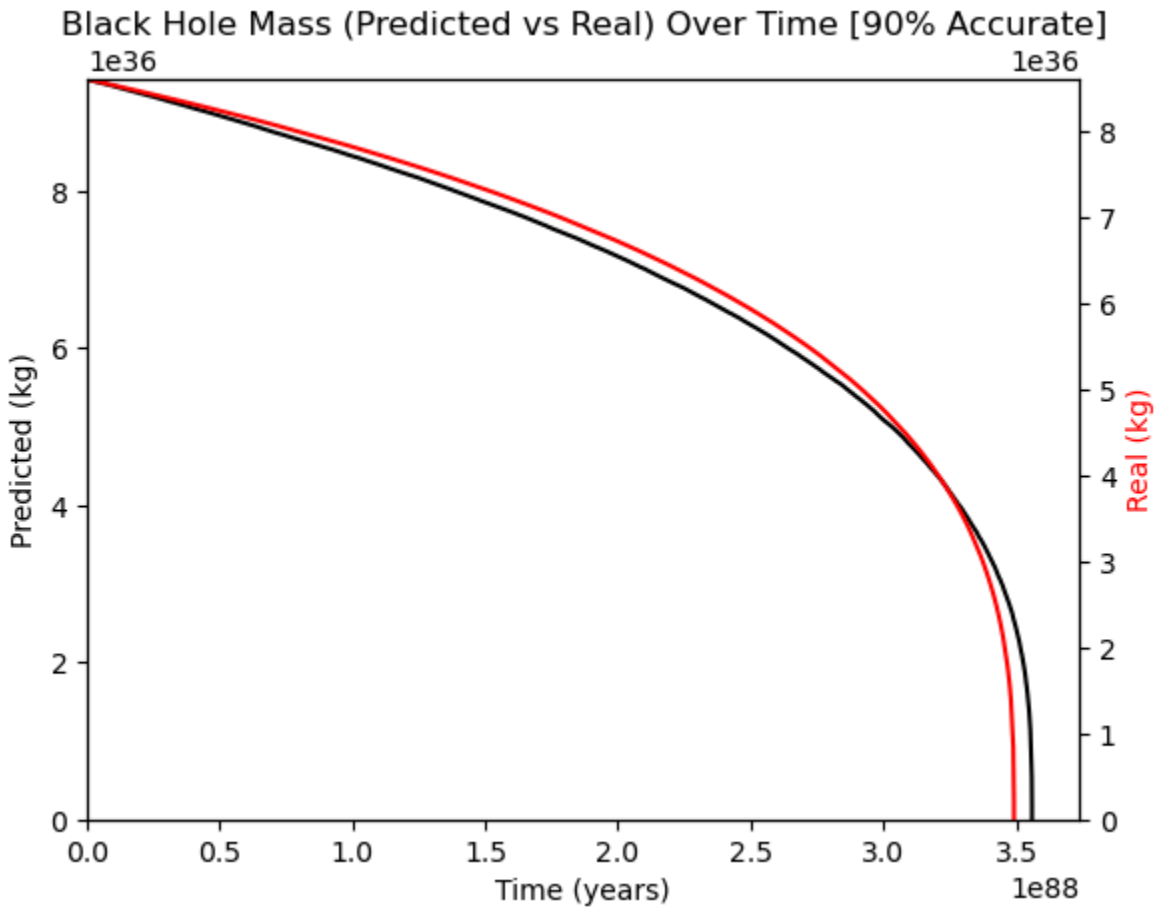
New best match: 7.63e+36kg

New best match: 8.48e+36kg

New best match: 9.42e+36kg

Closest match: ~9.424035e+36kg

Accuracy: 90%



Applications

- **Black Hole Information Paradox**
    - My model attempts to combat the black hole information paradox by demonstrating that information lost in a black hole can be recovered by its Hawking temperature emissions.
- **Broader Applications**
    - Black holes and Hawking radiation are some of the most interesting and least understood phenomena in our universe, potentially holding the secrets behind a theory of quantum gravity which could potentially revolutionize everything we understand about the universe and our place within it. A comprehensive link between quantum mechanics and gravity would open up possibilities in nuclear fusion physics, faster-than-light travel, and advanced technologies we have yet to grasp.

Conclusions

- In my engineering goal, I laid out my plans to create a model that visually and mathematically simulated Hawking radiation and predicted its initial mass and spin using its temperature data curve.
- While I did not implement a visual model or spin prediction, I accomplished my initial goal of creating an accurate Hawking radiation model that plotted mass/temperature over time, and a prediction model that estimated its initial mass based on that data. I also implemented spin conditions based on the Kerr solutions to general relativity, something I was unaware of when I first laid out my project methodology.

Recommendations

- **Simulation Model**
    - I initially planned to include a visual demonstration model, however I revoked that idea as it didn't contribute much to the overall goal of the project.
    - The graphing could also be cleaner. I primarily used matplotlib since it's beginner-friendly and very forgiving. Something like Plotly would have been ideal.
    - I would also like to potentially add parameters for charge (a property I was not aware black holes had until I had finished programming) using the Kerr-Newman solutions.
- **Prediction Model**
    - There are a lot of adjustments that need to be made to the prediction model; my initial outline had a much bigger scope for this aspect of the project, but it wasn't implemented due to time constraints.
    - **Spin**
        - The most glaring issue with my prediction model is that it only models one variable, mass. My reverse modeling algorithm only demonstrates how initial mass *and* spin could be calculated.
        - If only predicting one initial variable (mass), you only need one data point, the amount of time to decay, which can be deduced far more simply than by using the algorithm I created. An inverse modeling prediction algorithm is only necessary if tracking 2 variables, like mass and spin.
    - **Coarse-to-Fine Grid Search**
        - I also did not implement the gridsearch method I laid out in my plan, which would scan a large range of values until finding the most similar graph and search a similar number of values at a much smaller range surrounding that closest match. However, my prediction model can be adjusted manually to get finer and more accurate results.

Resources (Rationale)

Chu, Jennifer. “Physicists Observationally Confirm Hawking’s Black Hole Theorem for the First Time.” *MIT News*, 1 July 2021, https://news.mit.edu/2021/hawkings-black-hole-theorem-confirm-0701. *Accessed 28 Nov. 2025.*

NASA Jet Propulsion Laboratory. “How Scientists Captured the First Image of a Black Hole.” *JPL*, n.d., https://www.jpl.nasa.gov/edu/resources/teachable-moment/how-scientists-captured-the-first-image-of-a-black-hole/. *Accessed 28 Nov. 2025.*

Highfield, Roger. “A Brief History of Stephen Hawking’s Greatest Equation.” *Aeon*, n.d., https://aeon.co/essays/a-brief-history-of-stephen-hawkings-greatest-equation. *Accessed 28 Nov. 2025.*

Bernstein, Jeremy. “The Reluctant Father of Black Holes.” *Scientific American*, 1 Apr. 2007, https://www.scientificamerican.com/article/the-reluctant-father-of-black-holes-2007-04/. *Accessed 28 Nov. 2025.*

Fiveable. “Zero Entropy (AP Chemistry).” *Fiveable*, n.d., https://fiveable.me/key-terms/ap-chem/zero-entropy. *Accessed 28 Nov. 2025.*